

Context-Free Languages (CFL) - Complete Interview Guide

Scope: Context-Free Grammars, Derivations, Parse Trees, Ambiguity, Pushdown Automata, $\text{CFG} \leftrightarrow \text{PDA}$ equivalence, CNF, GNF, Pumping Lemma for CFLs, Closure & Decision properties, CYK algorithm, and Deterministic CFLs.

This is a TOC (Theory of Computation) topic. It sits **one level above Regular Languages** in the Chomsky hierarchy and is the theoretical backbone of every compiler and parser you will ever use.

0. The Big Picture (Read This First)

Before diving in, anchor yourself in the **Chomsky Hierarchy**. Everything below fits into this ladder:

Type	Language Class	Grammar	Machine	Example
Type 3	Regular	Regular Grammar	Finite Automaton (DFA/NFA)	a^*b^*
Type 2	Context-Free	CFG	Pushdown Automaton (PDA)	$a^n b^n$
Type 1	Context-Sensitive	CSG	Linear Bounded Automaton	$a^n b^n c^n$
Type 0	Recursively Enumerable	Unrestricted Grammar	Turing Machine	any computable set

Why the jump from Regular to Context-Free matters: A finite automaton has *finite memory* (its states). It cannot count arbitrarily high. To recognize $a^n b^n$ you must count the a s and match them against the b s - unbounded counting. A **stack** gives you exactly that unbounded-but-restricted memory. That single stack is the entire difference between Regular and Context-Free.

Keep this mental model: **CFL = Regular + one stack.**

1. Context-Free Grammar (CFG)

1. Overview

Definition. A Context-Free Grammar is a 4-tuple $\mathbf{G} = (\mathbf{V}, \mathbf{T}, \mathbf{P}, \mathbf{S})$ where:

- $\mathbf{V}$ = finite set of **variables** (non-terminals), e.g. S, A, B .
- $\mathbf{T}$ = finite set of **terminals** (the alphabet of the actual language), e.g. $a, b, 0, 1$. $\mathbf{V} \cap \mathbf{T} = \emptyset$.
- $\mathbf{P}$ = finite set of **production rules** of the form $A \rightarrow \alpha$, where $A \in \mathbf{V}$ (a **single** variable) and $\alpha \in (\mathbf{V} \cup \mathbf{T})^*$ (any string of variables/terminals, including ϵ).
- $\mathbf{S} \in \mathbf{V}$ = the **start symbol**.

The word "**context-free**" comes from the left-hand side: it is *always exactly one variable*, with **no surrounding context**. In a context-sensitive grammar you could write $aAb \rightarrow aab$ ("replace A only when between a and b"). CFGs forbid that - A can be replaced *regardless of what surrounds it*.

The language generated: $L(G) = \{ w \in T^* \mid S \Rightarrow^* w \}$ - all strings of terminals derivable from S.

Why it matters

- CFGs are the **formal specification of programming language syntax**. Every language spec (C, Java, Python, JSON, SQL) has a grammar, usually written in BNF/EBNF, which *is* a CFG.
- They describe **nested / recursive structure** - balanced parentheses, matched HTML tags, arithmetic expressions with precedence. Regular expressions provably cannot do this.
- They are the input to **parser generators** (yacc, bison, ANTLR).

Where it is used in real systems

- Compilers/interpreters:** the parser phase turns tokens into a parse tree using a CFG.
- Data formats:** JSON, XML, YAML, Protocol Buffers schemas.
- Query languages:** SQL, GraphQL grammars.
- Config & markup:** Markdown, regex engines' own syntax.
- Natural language processing:** phrase-structure grammars for sentence parsing.

Why interviewers ask about it

- Tests whether you understand the **limits of regular expressions** (a very common real-world confusion - people try to parse HTML/nested brackets with regex).
- Checks that you can **design a grammar** for a given language (a construction skill).
- It is the foundation for compiler-design questions and for the pumping-lemma / decidability theory questions that follow.

2. Core Idea

Intuition. A CFG is a set of *substitution rules*. You start with the start symbol and keep replacing variables with the right-hand sides of rules until only terminals remain. The set of all terminal strings you can reach is the language.

Real-world analogy. Think of a **Mad Libs template** or a set of **recipe expansions**. "A **<Sentence>** is a **<Noun>** followed by a **<Verb>**." "A **<Noun>** is 'cat' or 'dog'." You expand placeholders until you have a concrete sentence. The placeholders are variables; the concrete words are terminals.

Small example - balanced parentheses:

$$S \rightarrow (S) \mid S S \mid \epsilon$$

Read as: a balanced string is empty, OR a balanced string wrapped in one pair of parens, OR two balanced strings concatenated. This generates $()$, $(())$, $()()$, $(())()$, ...

Step-by-step derivation of $(())$:

$S \Rightarrow (S)$	[used $S \rightarrow (S)$]
$\Rightarrow ((S))$	[used $S \rightarrow (S)$ on inner S]
$\Rightarrow (())$	[used $S \rightarrow \epsilon$]

Classic example - $L = \{ a^n b^n \mid n \geq 0 \}$:

$$S \rightarrow a S b \mid \epsilon$$

The recursion $S \rightarrow a S b$ adds one a on the left and one matching b on the right *simultaneously*, guaranteeing equal counts. This is the canonical "regex can't do this, CFG can" example.

3. Important Subtopics

3.1 Terminals vs Variables

- **What:** Terminals are the actual output symbols; variables are intermediate placeholders that must all be expanded away.
- **Why:** A string is in the language only when it is **all terminals**. If any variable remains, it is a *sentential form*, not a finished string.
- **Interview angle:** "Is aSb in the language?" No - it contains a variable, it is a sentential form.

3.2 Sentential Forms

- **What:** Any string in $(V \cup T)^*$ derivable from S . It may contain both variables and terminals.
- **Left-sentential form:** obtained via leftmost derivations. **Right-sentential form:** via rightmost.
- **Interview angle:** Distinguishing "sentential form" from "sentence" (a fully-terminal string) is a common gotcha.

3.3 Recursion (the source of power)

- **Left-recursive:** $A \rightarrow A\alpha$ (variable reappears at the left end). Problematic for top-down parsers (infinite loop).
- **Right-recursive:** $A \rightarrow \alpha A$.
- **Why it matters:** Recursion is what lets a *finite* grammar generate an *infinite* language and nested structures. Removing left-recursion is a required preprocessing step for LL/recursive-descent parsers.

3.4 Designing a grammar (constructive skill)

- **What:** Given a language description, write the rules.
- **Technique:** Identify the *invariant* you must maintain (equal counts, palindrome symmetry, matching brackets) and encode it into recursive rules that add matched pieces together.
- **Interview angle:** e.g. "Grammar for equal number of a's and b's", "strings that are palindromes", " $a^i b^j c^k$ where $i = j$ or $j = k$ ".

3.5 CFG vs Regular Grammar

- **What:** A regular grammar restricts RHS to $A \rightarrow aB$ (right-linear) or $A \rightarrow Ba$ (left-linear), plus $A \rightarrow a / A \rightarrow \epsilon$. Every regular grammar is a CFG, so **every regular language is context-free**, but not vice versa.
- **Interview angle:** "Is every regular language context-free?" Yes. "Is every CFL regular?" No ($a^n b^n$ is the counterexample).

4. Real-World Example

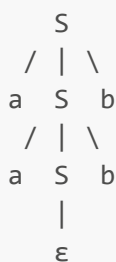
Arithmetic expression grammar (used inside every calculator / compiler):

```
E → E + T | T
T → T * F | F
F → ( E ) | id
```

When you type $2 + 3 * 4$ into a spreadsheet, calculator, or compiler, this grammar (or one like it) is what lets the parser know that $*$ binds tighter than $+$, so the result is 14 , not 20 . The **structure of the grammar encodes operator precedence and associativity** - a direct, practical payoff of CFG design.

5. Diagrams / Mental Models

CFG as an expansion tree (for $aabb$ from $S \rightarrow aSb \mid \epsilon$):



Reading leaves left-to-right (ignoring ϵ): $a a b b$

Mental model: A CFG is a **rewrite system**. State = current string of symbols. Move = pick a variable, replace it using any rule whose LHS is that variable. Accept = no variables left.

6. Common Interview Questions

Q1. What is a context-free grammar? Why "context-free"?

- *Answer:* A 4-tuple (V, T, P, S) whose productions have a single variable on the LHS. "Context-free" because that variable is rewritten regardless of surrounding symbols.
- *Key points:* single non-terminal on LHS; RHS any mix of terminals/variables.
- *Mistake:* Saying the RHS must be a single variable - only the **LHS** is restricted to one variable.

Q2. Give a CFG for $\{ a^n b^n \mid n \geq 1 \}$.

- *Answer:* $S \rightarrow aSb \mid ab$.
- *Key points:* recursion adds matched pairs; base case gives the minimum string.

- *Mistake:* Using $S \rightarrow ab$ alone (finite), or $S \rightarrow aSb \mid \epsilon$ when the spec says $n \geq 1$ (that allows the empty string).

Q3. Why can't a regular expression / DFA recognize $a^n b^n$?

- *Answer:* Finite memory. A DFA has finitely many states and cannot count arbitrarily large n . Proved via the pumping lemma for regular languages.
- *Mistake:* Saying "regex is not powerful enough" without the *finite-memory* / *cannot count* reason.

Q4. Is every regular language context-free?

- *Answer:* Yes. Regular grammars are a strict subset of CFGs.
- *Mistake:* Confusing the direction; the reverse is false.

Q5. Write a grammar for balanced parentheses.

- *Answer:* $S \rightarrow (S) \mid SS \mid \epsilon$.
- *Mistake:* Forgetting the SS rule (needed for $()()$) or the ϵ base case.

Q6. What is the difference between a terminal and a non-terminal?

- *Answer:* Terminals appear in final strings; non-terminals are placeholders expanded during derivation.
- *Mistake:* Thinking non-terminals can appear in the final accepted string.

Q7. What is a sentential form?

- *Answer:* Any string derivable from the start symbol that may still contain variables.
- *Mistake:* Equating it with a "sentence" (fully terminal string).

Q8. Give a CFG for palindromes over $\{a, b\}$.

- *Answer:* $S \rightarrow aSa \mid bSb \mid a \mid b \mid \epsilon$.
- *Key points:* symmetric rules add the same symbol on both ends; middle bases handle odd/even/empty.
- *Mistake:* Forgetting odd-length centers (a, b).

Q9. Grammar for equal number of a's and b's (any order)?

- *Answer:* $S \rightarrow aSb \mid bSa \mid SS \mid \epsilon$.
- *Mistake:* Only handling the $a^n b^n$ case; here order is free, so you need bSa and SS .

Q10. What class of languages do CFGs generate, and what machine recognizes them?

- *Answer:* Context-free languages; recognized by pushdown automata.
- *Mistake:* Saying "finite automata."

7. Deep-Dive Questions

D1. Can a CFG generate a non-context-free language? No - by definition the languages generated by CFGs are *exactly* the context-free languages. $a^n b^n c^n$ is not context-free, and indeed no CFG generates it (provable by the CFL pumping lemma).

D2. Are CFGs closed under substitution? Yes - CFLs are closed under substitution (replace each terminal with a CFL). This is a powerful closure property used to derive many others (union, concatenation, homomorphism).

D3. Given a CFG, is it decidable whether $L(G)$ is empty? Yes - mark all variables that can derive a terminal string (bottom-up); if S is marked, non-empty. This is a standard decidable property.

D4. Is it decidable whether two CFGs generate the same language? No - equivalence of CFGs is *undecidable*. (Contrast with DFAs, where it is decidable.)

D5. Why does natural language need more than CFGs? Some phenomena (cross-serial dependencies in Swiss-German, reduplication) are *mildly context-sensitive*, beyond CFG power - motivating formalisms like TAG and CCG. But CFGs capture the bulk of programming-language syntax perfectly.

2. Derivations, Leftmost & Rightmost Derivation

1. Overview

Definition. A **derivation** is a sequence of rule applications that transforms the start symbol into a string of terminals. We write $\alpha \Rightarrow \beta$ for " β is obtained from α by applying one production," and $\Rightarrow^*$ for zero-or-more steps.

- **Leftmost derivation (LMD):** at every step you replace the **leftmost** variable.
- **Rightmost derivation (RMD):** at every step you replace the **rightmost** variable.

Why it matters: Derivations are the *proof* that a string belongs to a language. Leftmost derivations correspond to **top-down parsing** (LL parsers); rightmost derivations (in reverse) correspond to **bottom-up parsing** (LR parsers). So this distinction maps directly onto the two major families of real parsers.

Where used: LL(1) recursive-descent parsers construct leftmost derivations; LR/LALR parsers (yacc, bison) construct rightmost derivations in reverse (reductions).

Why interviewers ask: To verify you can trace a derivation, connect LMD/RMD to parse trees, and understand that *a single parse tree can have exactly one LMD and one RMD*.

2. Core Idea

Intuition. Choosing which variable to expand next is a scheduling choice. The final string doesn't depend on the order, but the *derivation sequence* does. Fixing "always leftmost" or "always rightmost" removes that ordering freedom and gives a **canonical** derivation.

Analogy. Expanding an outline. "Leftmost" = always flesh out the earliest unfinished bullet before moving on (depth-first, left-to-right). "Rightmost" = always work on the latest unfinished bullet.

Example grammar: $E \rightarrow E + E \mid E * E \mid id$. Derive $id + id * id$.

Leftmost derivation:

```
E ⇒ E + E      (expand leftmost E)
  ⇒ id + E      (leftmost E → id)
  ⇒ id + E * E  (E → E * E)
```

```
⇒ id + id * E
⇒ id + id * id
```

Rightmost derivation:

```
E ⇒ E + E
  ⇒ E + E * E      (rightmost E → E * E)
  ⇒ E + E * id
  ⇒ E + id * id
  ⇒ id + id * id
```

Both yield the same string and (here) the same parse tree, but the *step order* differs.

3. Important Subtopics

3.1 One-step vs many-step derivation

- $\Rightarrow$ is one production application; $\Rightarrow^*$ is the reflexive-transitive closure (any number of steps, including zero).
- **Interview angle:** $S \Rightarrow^* S$ is always true (zero steps).

3.2 Leftmost derivation $\leftrightarrow$ top-down parsing

- LMD mirrors how a recursive-descent parser works: it expands the leftmost non-terminal, matching input left to right.
- **Why it matters:** LL(k) parsers *are* leftmost-derivation builders; understanding LMD explains why left-recursion breaks them.

3.3 Rightmost derivation $\leftrightarrow$ bottom-up parsing

- LR parsers build a rightmost derivation **in reverse** - each "reduce" step undoes the last step of a rightmost derivation. Hence LR = "Left-to-right scan, Rightmost derivation."
- **Interview angle:** "What does the R in LR stand for?" - Rightmost derivation (in reverse).

3.4 Derivation vs Parse Tree

- A parse tree **abstracts away the order** of expansion. Many derivations (different orders) can share one tree, but each tree has **exactly one** LMD and **exactly one** RMD.
- **Key theorem:** A grammar is ambiguous $\Leftrightarrow$ some string has ≥ 2 leftmost derivations (equivalently ≥ 2 parse trees, equivalently ≥ 2 rightmost derivations).

4. Real-World Example

When `gcc` or `javac` parses your source, an **LR parser** is (in reverse) constructing a rightmost derivation of your program from the language grammar. Each shift pushes a token; each reduce replaces a right-hand side with its non-terminal - literally running a rightmost derivation backward. If you have ever seen a "shift/reduce conflict" from bison, that is this machinery talking.

5. Diagrams / Mental Models

Parse tree (one) $\begin{array}{l} \rightarrow \text{exactly one Leftmost derivation} \\ \rightarrow \text{exactly one Rightmost derivation} \end{array}$

Ambiguous string $\rightarrow \geq 2$ parse trees $\rightarrow \geq 2$ LMDs $\rightarrow \geq 2$ RMDs

Counting rule of thumb: number of parse trees = number of distinct LMDs = number of distinct RMDs.

6. Common Interview Questions

Q1. Define a derivation. A sequence of production applications from S to a terminal string. *Mistake:* stopping while variables remain.

Q2. Leftmost vs rightmost derivation? LMD always expands the leftmost variable; RMD the rightmost. *Mistake:* thinking they produce different strings - they produce the same string, different step orders.

Q3. Does LMD or RMD change the generated language? No. The set of strings is identical; only the derivation order changes. *Mistake:* claiming one is more powerful.

Q4. How many LMDs does a single parse tree have? Exactly one. Same for RMD. *Mistake:* saying "many."

Q5. How do you detect ambiguity using derivations? Find a string with two distinct leftmost (or rightmost) derivations. *Mistake:* comparing an LMD with an RMD of the same tree and calling it ambiguity - those are always different orders even for unambiguous grammars.

Q6. Which parser uses leftmost derivations? Top-down / LL / recursive-descent. *Mistake:* saying LR.

Q7. Which parser uses rightmost derivations (in reverse)? Bottom-up / LR / LALR.

Q8. Trace a leftmost derivation of $id+id$ for $E \rightarrow E+E \mid id$. $E \Rightarrow E+E \Rightarrow id+E \Rightarrow id+id$.

Q9. Is $S \Rightarrow^* S$ valid? Yes, zero steps. *Mistake:* saying no.

Q10. If a string has one LMD but you can also write an RMD, is the grammar ambiguous? No - every derivable string has both an LMD and an RMD. Ambiguity needs *two* LMDs (or two RMDs), not one of each.

7. Deep-Dive Questions

D1. Why do LR parsers use *reverse* rightmost derivations? Because they read input left-to-right and reduce as soon as a complete right-hand side (handle) appears on the stack - each reduction is the inverse of the last step of a rightmost derivation.

D2. Can two different parse trees give the same leftmost derivation? No - the LMD uniquely determines and is determined by the parse tree.

D3. Is finding *whether* a leftmost derivation exists decidable? Yes - it is just the membership problem, decidable for CFGs (e.g., by CYK).

D4. Relationship between derivation length and string length in CNF? In Chomsky Normal Form, deriving a terminal string of length n takes exactly $2n - 1$ steps ($n - 1$ binary rules + n terminal rules). This fact underpins the CYK complexity analysis.

D5. Why does left recursion break recursive-descent (LMD) parsing but not LR (RMD)? Recursive descent expands the leftmost variable first; with $A \rightarrow A\alpha$ it recurses on A forever without consuming input. LR defers the decision until it has seen the handle, so left recursion is fine (even preferred) for LR.

3. Parse Trees (Derivation Trees)

1. Overview

Definition. A **parse tree** (derivation tree) for a CFG is an ordered, rooted tree where:

- the **root** is the start symbol,
- each **internal node** is a variable, and its children (left-to-right) are exactly the symbols on the RHS of some production applied to it,
- each **leaf** is a terminal or ϵ ,
- reading the leaves left to right (the **yield** / frontier) gives the derived string.

Why it matters: The parse tree is the **structural meaning** of a string. Compilers don't keep the derivation sequence; they keep the tree (then simplify it into an **AST**) because the tree encodes *grouping*, *precedence*, and *scope*.

Where used: Every compiler's parser output; XML/JSON DOM trees; expression evaluators; syntax highlighters; linters.

Why interviewers ask: Parse trees are the cleanest way to see ambiguity, and they connect grammar to semantics (precedence, associativity).

2. Core Idea

Intuition. A derivation is a *movie* (ordered steps); a parse tree is the *photo* (final structure with order forgotten). The tree throws away "which variable I expanded first" but keeps "which rule produced which children."

Analogy. An organizational chart. The CEO (start symbol) has departments (variables) that have sub-teams, down to individual employees (terminals). It doesn't matter which department you drew first; the reporting structure is what counts.

Example for $E \rightarrow E + E \mid E * E \mid id$ and string $id + id * id$:



```

      |   |
    id   id

```

The yield is `id + id * id`. This particular tree groups `id*id` together, then adds `id` - i.e. `id + (id*id)`.

3. Important Subtopics

3.1 Yield / Frontier

- **What:** The left-to-right concatenation of leaf labels.
- **Why:** It is the string the tree "represents." Two trees with the same yield but different shapes $\Rightarrow$ ambiguity.

3.2 Parse tree $\leftrightarrow$ Derivation correspondence

- Each parse tree $\leftrightarrow$ exactly one LMD and one RMD.
- **Interview angle:** "How many parse trees can a string have?" Possibly many (that's ambiguity); each corresponds to a distinct LMD.

3.3 Abstract Syntax Tree (AST) vs Parse/Concrete Syntax Tree

- The **parse tree** (concrete syntax tree) includes *every* grammar symbol, even ones like parentheses and single-child chains ($E \rightarrow T \rightarrow F$).
- The **AST** strips redundant nodes, keeping only semantically meaningful structure.
- **Interview angle:** Compilers build the AST from the parse tree; know the difference.

3.4 Subtrees and Structural Meaning

- A subtree rooted at a variable derives a substring; this "constituent" structure is what gives operators their precedence and scoping.

4. Real-World Example

The **DOM (Document Object Model)** in a browser is essentially the parse tree of your HTML according to the HTML grammar. `<div><p>hi</p></div>` becomes a tree with `div` as parent of `p` as parent of the text node `hi`. JavaScript's `document.querySelector` walks this parse-tree-derived structure.

5. Diagrams / Mental Models

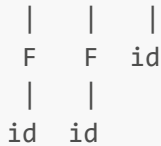
Precedence encoded by tree depth - deeper = evaluated first:

Grammar with precedence: $E \rightarrow E + T \mid T ; \quad T \rightarrow T * F \mid F ; \quad F \rightarrow id$
 Parse tree of `id + id * id`:

```

      E
    / | \
  E  +  T
  |   /|\
  T  T * F

```



The '*' subtree sits BELOW the '+', so multiplication binds tighter.

6. Common Interview Questions

Q1. What is a parse tree? A rooted ordered tree: root = start symbol, internal nodes = variables expanded by a rule, leaves = terminals; yield = derived string. *Mistake:* forgetting the leaves-in-order = string property.

Q2. Parse tree vs derivation? Tree abstracts away step order; a derivation is an ordered sequence. One tree $\leftrightarrow$ one LMD $\leftrightarrow$ one RMD. *Mistake:* thinking they are the same.

Q3. How does a parse tree reveal ambiguity? Two distinct parse trees for one string $\Rightarrow$ ambiguous grammar.

Q4. What is the yield of a parse tree? The string formed by its leaves left to right.

Q5. Parse tree vs AST? Parse tree keeps every grammar symbol; AST keeps only semantic essentials. *Mistake:* using them interchangeably.

Q6. Can an internal node be a terminal? No - internal nodes are variables; terminals are always leaves.

Q7. How many parse trees does an unambiguous grammar give per string? Exactly one (for strings in the language).

Q8. Draw parse tree for `id+id*id` with a precedence grammar. (See section 5.) Multiplication subtree lower.

Q9. Why do compilers build trees, not keep derivations? Trees encode structure/precedence/scope directly and are order-independent - ideal for semantic analysis.

Q10. Can two different strings share a parse tree? No - the yield is fixed by the tree, so one tree = one string.

7. Deep-Dive Questions

D1. How is precedence "baked into" a parse tree? By stratifying the grammar into levels ($E \rightarrow T \rightarrow F$). Higher-precedence operators appear at lower grammar levels, hence deeper in the tree, hence evaluated first.

D2. Can you always convert a parse tree to an AST algorithmically? Yes - during parsing you emit AST nodes for meaningful reductions and skip "pass-through" single-child productions.

D3. What is a *constituent* and why does it matter for NLP? A subtree; in NLP constituents are phrases (noun phrase, verb phrase) - the tree encodes grammatical structure used for meaning extraction.

D4. If two parse trees have the same LMD, are they equal? Yes - the LMD determines the tree uniquely.

D5. Does adding parentheses to a grammar remove ambiguity? It can - explicit bracketing forces a unique grouping (that is exactly what `( E )` in the expression grammar does).

4. Ambiguous Grammars & Removing Ambiguity

1. Overview

Definition. A CFG **G** is **ambiguous** if there exists at least one string $w \in L(G)$ that has **two or more distinct parse trees** (equivalently, two or more leftmost derivations, or two or more rightmost derivations).

Key subtlety: Ambiguity is a property of the **grammar**, not the language. A language is **inherently ambiguous** if every grammar for it is ambiguous.

Why it matters: Ambiguity means a string has **more than one structural interpretation** - and therefore more than one *meaning*. $2 + 3 * 4$ under an ambiguous grammar could mean 14 or 20. Compilers must have a single, deterministic interpretation, so ambiguous grammars are a real bug.

Where used / seen: operator-precedence issues, the classic **dangling-else** problem, shift/reduce conflicts in yacc/bison.

Why interviewers ask: It tests deep understanding - the difference between grammar and language, how to *diagnose* ambiguity, and how to *fix* it (precedence, associativity, layering).

2. Core Idea

Intuition. Ambiguity = the grammar gives the parser a *genuine choice* at some point that leads to two different valid structures for the same input. Removing ambiguity = removing that choice by encoding precedence/associativity into the grammar's shape.

Analogy. The English sentence "*I saw the man with the telescope.*" Did I use the telescope, or did the man have one? Two parse trees, two meanings - natural language is riddled with ambiguity. Programming languages must not be.

Small example - the ambiguous expression grammar:

```
E → E + E | E * E | id
```

$id + id * id$ has **two** parse trees:

- Tree A: $(id + id) * id \rightarrow$ treats $+$ as tighter.
- Tree B: $id + (id * id) \rightarrow$ treats $*$ as tighter.

Both are valid derivations $\Rightarrow$ ambiguous.

3. Important Subtopics

3.1 Detecting ambiguity

- **What:** Find one string with two parse trees / two LMDs.

- **Why:** There is **no algorithm** to decide ambiguity in general (it is *undecidable*), so you reason by finding a witness string.
- **Interview angle:** "Prove this grammar is ambiguous" = exhibit two trees for one string.

3.2 Removing ambiguity via precedence & associativity (layering)

- **What:** Rewrite into a stratified grammar with one level per precedence tier and recursion direction chosen for associativity.

```
E → E + T | T      (+ is left-associative, lowest precedence)
T → T * F | F      (* is left-associative, higher precedence)
F → ( E ) | id     (highest: atoms and parentheses)
```

- **Why:** Left recursion ($E \rightarrow E + T$) $\Rightarrow$ left associativity; higher-precedence operators placed at deeper levels $\Rightarrow$ they bind first. Now `id + id * id` has exactly one tree.
- **Interview angle:** Very commonly asked: "Rewrite this ambiguous expression grammar to be unambiguous."

3.3 The Dangling-Else problem

- **What:** $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid \text{other}$. The string `if e1 then if e2 then s1 else s2` has two parses (does `else` bind to the inner or outer `if`?).
- **Fix:** Introduce *matched* vs *unmatched* statements so `else` always binds to the nearest unmatched `if`:

```
S → M | U
M → if E then M else M | other
U → if E then S | if E then M else U
```

- **Interview angle:** A classic. Know both the problem and the matched/unmatched fix. (In practice, yacc resolves it by defaulting to *shift*, binding else to the nearest if.)

3.4 Inherently ambiguous languages

- **What:** Some CFLs (e.g. $L = \{ a^i b^j c^k \mid i=j \text{ OR } j=k \}$) have **no** unambiguous grammar at all.
- **Why:** Because the two "reasons" a string can be in the language overlap on strings where $i=j=k$, forcing two derivations.
- **Interview angle:** "Can every ambiguous grammar be made unambiguous?" No - if the language is inherently ambiguous, ambiguity is unavoidable.

3.5 Ambiguity is undecidable

- **What:** No algorithm decides, for an arbitrary CFG, whether it is ambiguous.
- **Interview angle:** Contrasts with membership/emptiness (decidable).

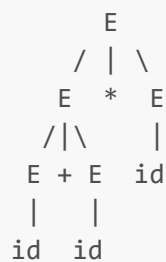
4. Real-World Example

yacc/bison shift-reduce conflicts. When you write an ambiguous grammar for a parser generator, it reports "shift/reduce conflict." That is the tool telling you two parse trees are possible. Real grammars resolve this with `%left`, `%right`, `%prec` directives (declaring precedence/associativity) - the tool then *breaks the tie* deterministically instead of you rewriting the whole grammar. The dangling-else is the textbook shift/reduce conflict, resolved by preferring shift.

5. Diagrams / Mental Models

Ambiguous grammar $E \rightarrow E+E \mid E * E \mid id$, string $id+id*id$:

Tree A (+ first)



Tree B (* first)



Two trees for one string $\Rightarrow$ AMBIGUOUS.

Fix by layering into $E \rightarrow E+T$, $T \rightarrow T * F$, $F \rightarrow id \Rightarrow$ one tree only.

6. Common Interview Questions

Q1. Define an ambiguous grammar. A grammar where some string has ≥ 2 parse trees / LMDs / RMDs.

Mistake: saying " ≥ 2 derivations" without specifying *leftmost* - every string has both an LMD and RMD; that alone isn't ambiguity.

Q2. Ambiguity: property of grammar or language? Grammar. A language is *inherently ambiguous* only if *all* its grammars are ambiguous. *Mistake:* conflating the two.

Q3. Make $E \rightarrow E + E \mid E * E \mid id$ unambiguous. Use the layered $E \rightarrow E+T$, $T \rightarrow T * F$, $F \rightarrow (E) \mid id$. *Mistake:* forgetting parentheses or getting associativity direction wrong.

Q4. What is the dangling-else problem and how do you fix it? Ambiguity of else-binding; fix with matched/unmatched non-terminals (or shift preference).

Q5. Is ambiguity decidable? No - undecidable in general. *Mistake:* claiming there's an algorithm.

Q6. Can every ambiguous grammar be disambiguated? Not always - inherently ambiguous languages can't.

Q7. Give an inherently ambiguous language. $\{ a^i b^j c^k \mid i=j \text{ or } j=k \}$.

Q8. How does left vs right recursion affect associativity? Left recursion $\Rightarrow$ left associative; right recursion $\Rightarrow$ right associative.

Q9. How do parser generators handle ambiguity? Precedence/associativity declarations that resolve shift/reduce and reduce/reduce conflicts deterministically.

Q10. Does ambiguity change the *language*? No - the set of strings is the same; only the number of structures per string differs.

7. Deep-Dive Questions

D1. Why is ambiguity undecidable? It reduces from the Post Correspondence Problem - one can build a grammar that is ambiguous iff a PCP instance has a solution.

D2. Why does layering by precedence remove ambiguity? Each precedence level has a unique non-terminal, so the parser has no choice about *where* an operator attaches - the grammar structure forces a single grouping.

D3. Is $a + b + c$ associativity forced by the grammar or by semantics? By the grammar's recursion direction. $E \rightarrow E + T$ forces $(a+b)+c$ (left). To get right-assoc $a+(b+c)$ use $E \rightarrow T + E$.

D4. Are unambiguous grammars closed under union? No - the union of two languages with unambiguous grammars can be inherently ambiguous.

D5. Is every deterministic CFL unambiguous? Yes - every DCFL has an unambiguous grammar (LR grammar). But the converse fails: some unambiguous CFLs are not deterministic.

8. Comparison Table

Aspect	Ambiguous Grammar	Unambiguous Grammar
Parse trees per string	≥ 2 for some string	Exactly 1 for every string
Meaning	Multiple interpretations	Single interpretation
Suitable for compilers	No	Yes
Detection	Undecidable in general	-
Example	$E \rightarrow E+E \mid id$	$E \rightarrow E+T; T \rightarrow id$

5. Pushdown Automata (PDA)

1. Overview

Definition. A **Pushdown Automaton** is a finite automaton augmented with a **stack** (unbounded LIFO memory). Formally a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$:

- Q = finite states, Σ = input alphabet, Γ = **stack alphabet**,
- $\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow$ finite subsets of $Q \times \Gamma^*$ (the transition function),
- q_0 = start state, Z_0 = initial stack symbol, F = accepting states.

A move reads (current state, input symbol or ϵ , top-of-stack symbol) and produces (new state, string to push in place of the top symbol).

Why it matters: The PDA is the **machine model for CFLs** - exactly as the DFA is for regular languages. **L is context-free $\Leftrightarrow$ some PDA accepts L.** The stack provides the *unbounded counting/matching* memory a

DFA lacks.

Where used: the theoretical model behind every stack-based parser; recursive-descent parsing uses the call stack as its PDA stack; matching-bracket checkers; expression evaluators (shunting-yard uses a stack).

Why interviewers ask: To confirm you understand *why* a stack lifts you from regular to context-free, the two acceptance modes, and the crucial fact that **nondeterministic PDAs are strictly more powerful than deterministic ones**.

2. Core Idea

Intuition. The finite control decides *what to do*; the stack *remembers how much*. To accept $a^n b^n$: push a marker for each a , then pop one marker for each b . If the stack empties exactly as input ends, counts matched.

Analogy. A stack of plates while reading a to-do list. Every "add X" pushes a plate; every "resolve X" pops one. If you finish the list with zero plates left, everything was balanced. Function call stacks work identically - each call pushes a frame, each return pops one.

Small example - PDA for $\{ a^n b^n \mid n \geq 1 \}$ (accept by empty stack):

```
Start: stack has  $Z_0$ .
On 'a', push A:       $(q_0, a, Z_0) \rightarrow (q_0, A Z_0)$ 
                    $(q_0, a, A) \rightarrow (q_0, A A)$ 
On first 'b', start popping:  $(q_0, b, A) \rightarrow (q_1, \epsilon)$ 
On more 'b', keep popping:   $(q_1, b, A) \rightarrow (q_1, \epsilon)$ 
End:  $(q_1, \epsilon, Z_0) \rightarrow (q_1, \epsilon)$  [ $Z_0$  exposed, input done  $\rightarrow$  accept]
```

Reading $aabb$: push A, push A (stack AAZ_0), read b pop A, read b pop A (stack Z_0), input ends with Z_0 on top $\Rightarrow$ accept.

3. Important Subtopics

3.1 Instantaneous Description (ID) & moves

- **What:** A snapshot $(q, w, \gamma) = (\text{current state, remaining input, stack contents})$. The $\vdash$ relation denotes one move.
- **Why:** IDs are how you formally trace/prove a PDA run.

3.2 Two acceptance modes: final state vs empty stack

- **Accept by final state:** input consumed and PDA is in some state in F .
- **Accept by empty stack:** input consumed and stack is empty (F irrelevant).
- **Key fact:** the two modes are **equivalent in power** - any language accepted by one can be accepted by the other (with a simple construction adding a bottom marker / cleanup state).
- **Interview angle:** "Are the two acceptance criteria equally powerful?" Yes.

3.3 Nondeterministic PDA (NPDA) vs Deterministic PDA (DPDA)

- **What:** An NPDA may have multiple moves for a configuration and can make ϵ -moves freely; a DPDA has at most one applicable move.
- **Critical fact: NPDA is strictly more powerful than DPDA.** NPDAs recognize *all* CFLs; DPDAs recognize only the **deterministic CFLs (DCFLs)**, a proper subset.
- **Interview angle:** "For PDAs, is nondeterminism more powerful than determinism?" **Yes** - unlike finite automata where NFA = DFA. This is one of the most important contrasts in the whole topic.
- **Example needing nondeterminism:** even-length palindromes $\{ w w^R \}$ - a DPDA can't know where the middle is; an NPDA guesses.

3.4 ϵ -moves and stack operations

- **push:** replace top X with YX (or AX etc.).
- **pop:** replace top X with ϵ .
- **no-op / state change:** replace X with X.
- ϵ -transitions let the PDA move without consuming input - essential for nondeterministic guessing.

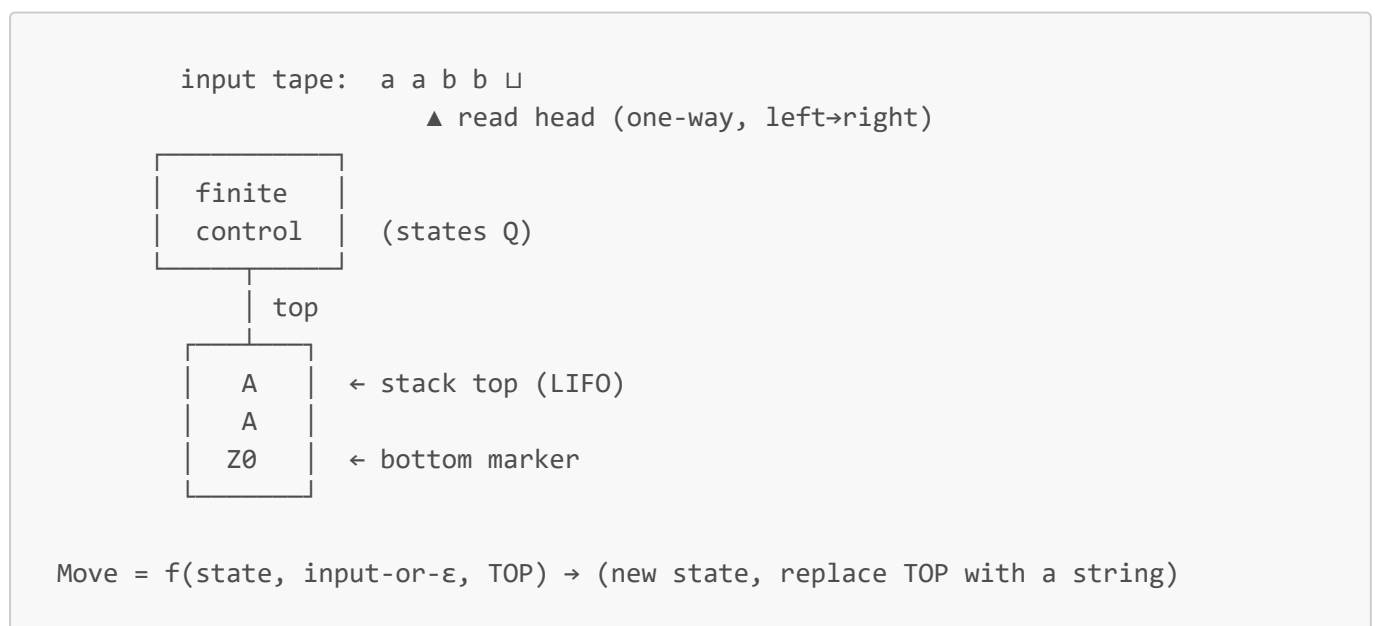
3.5 The stack is the only unbounded memory

- **What:** Access is strictly LIFO - you only ever see/modify the top. This restriction is exactly what keeps PDAs from being as powerful as Turing machines (which have unrestricted tape access).
- **Interview angle:** "Why can't a PDA recognize $a^n b^n c^n$?" One stack can match a's with b's or b's with c's, but not both simultaneously - LIFO access prevents it.

4. Real-World Example

Recursive-descent parsing and matching brackets. When your editor checks that every $\{$, $($, $[$ is properly closed, it runs a PDA: push on open bracket, pop-and-match on close bracket, accept if the stack empties. The same principle drives the **call stack** of a running program - each function call pushes an activation record, each return pops it; stack overflow is literally the PDA stack exceeding its (real, finite) bound.

5. Diagrams / Mental Models



6. Common Interview Questions

Q1. What is a PDA? A finite automaton + a stack; the machine model for CFLs. *Mistake:* forgetting the stack is *unbounded*.

Q2. Why is a PDA more powerful than a DFA? The unbounded stack allows counting/matching (e.g. $a^n b^n$), which finite memory can't.

Q3. What are the two acceptance modes and are they equivalent? Final state and empty stack; equivalent in power. *Mistake:* thinking one is stronger.

Q4. Are NPDA and DPDA equivalent in power? No - NPDA is strictly more powerful; DPDA recognizes only DCFLs. *Mistake:* carrying over "NFA = DFA" - it is FALSE for PDAs.

Q5. Give a PDA for $a^n b^n$. Push on a, pop on b, accept when balanced (see section 2).

Q6. Why can't a PDA recognize $a^n b^n c^n$? A single LIFO stack can match only one pair of counts at a time.

Q7. What does an instantaneous description contain? (state, remaining input, stack contents).

Q8. What operations can a PDA do on the stack per move? Push, pop, or leave the top (replace top by a string over Γ). Only the top is accessible.

Q9. Give a language needing an NPDA (not doable by DPDA). $\{ w w^R \}$, even-length palindromes.

Q10. What class of languages do PDAs recognize? Exactly the context-free languages.

7. Deep-Dive Questions

D1. Prove the two acceptance modes are equivalent. From empty-stack to final-state: add a new bottom marker X_0 below Z_0 and a new final state reached by ϵ -move when X_0 is exposed. From final-state to empty-stack: after entering a final state, ϵ -pop everything. Standard constructions.

D2. Why exactly is NPDA > DPDA? Because acceptance sometimes requires *guessing* the future (e.g. the midpoint of a palindrome) - a deterministic machine can't backtrack the single one-way input. DCFLs are precisely those needing no such guess.

D3. Is the class of DPDA languages closed under complement? Under union? Complement: **yes** (DCFLs are closed under complement). Union: **no** (DCFLs not closed under union). This asymmetry is a favorite trick question.

D4. Two-stack PDA - how powerful? A PDA with **two** stacks is equivalent to a **Turing machine** (one stack for the left of the tape, one for the right). This shows the stack count is the knob controlling computational power.

D5. Does adding a counter instead of a stack change power? A one-counter automaton is weaker than a full PDA (a counter is a stack over a one-symbol alphabet plus a bottom marker). It recognizes only a subclass.

8. Comparison Table

Feature	DFA/NFA	PDA (NPDA)	DPDA
Extra memory	none	one stack (unbounded)	one stack (unbounded)

Feature	DFA/NFA	PDA (NPDA)	DPDA
Language class	Regular	All CFLs	DCFLs ($\subsetneq$ CFL)
Determinism = Nondeterminism?	Yes (NFA=DFA)	-	No (DPDA $\subsetneq$ NPDA)
Closed under complement	Yes	No	Yes
Closed under union	Yes	Yes	No
Recognizes ww^R (palindrome)	No	Yes	No

6. CFG $\leftrightarrow$ PDA Equivalence

1. Overview

Statement. A language L is context-free **if and only if** some PDA (by empty stack, equivalently by final state, using an NPDA) accepts it. Formally:

For every CFG G there is a PDA M with $L(M) = L(G)$, and for every PDA M there is a CFG G with $L(G) = L(M)$.

Why it matters: This is the **central duality of CFLs** - the *generator* (grammar) and the *recognizer* (machine) describe exactly the same class. It is the direct analogue of "regular expression $\Leftrightarrow$ finite automaton." It lets you switch representations depending on the task: grammars for specifying syntax, PDAs for reasoning about recognition.

Where used: Parser theory - LL/LR parsers are essentially structured PDAs built from a grammar. The construction "grammar $\rightarrow$ PDA" is exactly how a top-down parser is derived.

Why interviewers ask: It's a defining theorem of the topic. They want the *idea* of both directions, not necessarily every formal detail.

2. Core Idea

Direction 1: CFG $\rightarrow$ PDA (the easy, must-know direction). Build a **single-state PDA** that keeps a **sentential form on the stack** and simulates leftmost derivations:

1. Start by pushing the start symbol S onto the stack (over Z_0).
2. **If top of stack is a variable A :** ϵ -move - pop A and push the RHS of some production $A \rightarrow \alpha$ (nondeterministically choose which). (Push α reversed so the leftmost symbol ends up on top.)
3. **If top of stack is a terminal a :** read a from input and pop it (they must match).
4. **Accept by empty stack** when input is exhausted.

The stack always holds the *remaining part* of a leftmost-derivation sentential form. Nondeterminism handles the choice of production.

Direction 2: PDA $\rightarrow$ CFG (the harder direction, know it exists). Given a PDA (accepting by empty stack), build variables of the form $[p, X, q]$ meaning "the PDA can go from state p to state q while popping exactly

the symbol X off the stack (net)." Productions simulate the PDA's moves. The start symbol derives every accepted string. This "triple construction" proves the reverse inclusion.

Analogy. CFG→PDA is like turning a *recipe* (rules) into a *cook* (a machine that follows rules using a stack of pending steps). PDA→CFG is reverse-engineering the recipe from watching the cook.

3. Important Subtopics

3.1 The CFG → PDA construction (single-state, LL-style / top-down)

- **What:** Simulate leftmost derivations with the sentential form on the stack.
- **Why it matters:** This is the theoretical basis of **top-down / predictive parsing**.
- **Interview angle:** "How do you convert a grammar to a PDA?" - describe the three move types above.

3.2 The PDA → CFG construction (triple / $[pXq]$ variables)

- **What:** Variables encode "start state, stack symbol popped, end state."
- **Why it matters:** Proves the full equivalence (both directions).
- **Interview angle:** You usually just need to state the idea and the $[p \times q]$ meaning.

3.3 Empty-stack vs final-state acceptance in the proof

- The CFG→PDA construction most naturally yields an **empty-stack** PDA. Because the two acceptance modes are equivalent, this suffices for full generality.

3.4 Why the PDA must be nondeterministic

- The CFG→PDA construction *chooses* a production when a variable is on top. For an ambiguous or merely nondeterministic grammar, that choice is a guess $\Rightarrow$ the resulting PDA is an **NPDA**. This is why CFGs correspond to *nondeterministic* PDAs, and why DPDA's (a strict subset) match only DCFLs.

4. Real-World Example

A **recursive-descent parser** is the CFG→PDA construction made concrete: the program's **call stack** plays the role of the PDA stack. Calling the procedure for non-terminal A = pushing A 's expansion; matching a token = the "pop terminal, read input" move. When you write a hand-rolled parser, you are literally building the PDA that the equivalence theorem guarantees exists.

5. Diagrams / Mental Models

CFG → PDA (single state q):

```

 $\delta(q, \epsilon, A) = \{ (q, \alpha) \text{ for each rule } A \rightarrow \alpha \}$  // expand a variable
 $\delta(q, a, a) = \{ (q, \epsilon) \}$  // match a terminal
Start: push  $S$ . Accept: empty stack.

```

Trace of aabb with $S \rightarrow aSb \mid ab$:

```

stack (top→): S          input aabb

```

expand $S \rightarrow aSb$: $a \ S \ b$ read $a \rightarrow S \ b$ (matched a)
 expand $S \rightarrow ab$: $a \ b$ read... etc, eventually stack empty $\Rightarrow$ accept

6. Common Interview Questions

Q1. State the CFG-PDA equivalence. L is context-free iff some (nondeterministic) PDA accepts it; every CFG has an equivalent PDA and vice versa.

Q2. Outline CFG $\rightarrow$ PDA. Single-state PDA: push S; expand a top variable by a production via ϵ -move; match a top terminal against input; accept by empty stack. *Mistake:* forgetting to push the RHS reversed so the leftmost symbol is on top.

Q3. How many states does the standard CFG $\rightarrow$ PDA construction need? Just **one** (plus optionally a start-up move). *Mistake:* overcomplicating with many states.

Q4. Which acceptance mode does the construction use? Empty stack (equivalent to final state).

Q5. Why is the resulting PDA nondeterministic? Because it guesses which production to apply for a top variable.

Q6. Sketch PDA $\rightarrow$ CFG. Variables $[p \ X \ q]$ = "go $p \rightarrow q$ net-popping X"; productions mimic PDA moves; start symbol derives all accepted strings.

Q7. Does the equivalence hold for deterministic PDAs? No - DPDAs correspond only to DCFLs, a strict subset. The full equivalence is for NPDAs.

Q8. What does the stack hold during the CFG $\rightarrow$ PDA simulation? The suffix of the current leftmost-derivation sentential form still to be matched.

Q9. Is the CFG $\rightarrow$ PDA simulation top-down or bottom-up? Top-down (it mirrors leftmost derivations). A different (bottom-up) construction mirrors LR parsing.

Q10. Real-world manifestation of CFG $\rightarrow$ PDA? Recursive-descent parsing using the call stack.

7. Deep-Dive Questions

D1. Why does the $[pXq]$ variable capture the right thing? Because a CFL derivation corresponds to a "balanced" push-then-pop sequence on the stack; the triple records where the net pop of one symbol starts and ends, matching the recursive structure of derivations.

D2. Can the CFG $\rightarrow$ PDA construction be made deterministic for LL(1) grammars? Yes - LL(1) grammars produce a *deterministic* top-down PDA (the parse table removes the guessing). That's precisely why LL(1) grammars are parseable without backtracking.

D3. What is the bottom-up analogue of CFG $\rightarrow$ PDA? The LR construction - a PDA that shifts input onto the stack and reduces handles, simulating reverse rightmost derivations.

D4. Does equivalence give an efficient parser automatically? No - the raw NPDA may need exponential backtracking. Efficient parsing needs restricted grammars (LL, LR) or the polynomial CYK/Earley algorithms.

D5. Is the two-way equivalence constructive? Yes - both directions are explicit algorithms, so you can mechanically convert either representation to the other.

7. Chomsky Normal Form (CNF)

1. Overview

Definition. A CFG is in **Chomsky Normal Form** if every production has one of these forms:

- $A \rightarrow BC$ (exactly two variables on the right), or
- $A \rightarrow a$ (exactly one terminal), and optionally
- $S \rightarrow \epsilon$ (only if $\epsilon \in L$, and S the start symbol never appears on any RHS).

Why it matters: CNF gives every production a **fixed, simple shape**, which makes algorithms clean and analysis tractable. Two headline payoffs:

1. The **CYK parsing algorithm** requires CNF and runs in $O(n^3)$.
2. The **parse tree becomes (almost) binary**, so a terminal string of length n derives in exactly $2n - 1$ steps - a fact used to bound derivation lengths and prove decidability results.

Where used: CYK membership testing; theoretical proofs (pumping lemma constant, decidability of membership); many textbook constructions require CNF as a preprocessing step.

Why interviewers ask: Converting a grammar to CNF is a classic mechanical exercise, and CNF underpins CYK, a very commonly asked algorithm.

2. Core Idea

Intuition. CNF is a *normalized* grammar - you strip away all the "irregular" production shapes (ϵ -productions, unit productions, long RHS, terminals mixed with variables) until only two clean forms remain. The language is preserved; only the grammar's shape changes.

Analogy. Like converting arbitrary boolean formulas to CNF (conjunctive normal form) in logic, or normalizing a database schema - same information, standardized structure that algorithms can rely on.

The 5-step conversion procedure (memorize this order):

```
START (optional): add fresh start  $S_0 \rightarrow S$  so start symbol never on any RHS.
1. TERM: replace terminals in long rules with new variables ( $A \rightarrow aB \Rightarrow A \rightarrow Xa B$ ,  $Xa \rightarrow a$ ).
2. BIN : break RHS with >2 symbols into a chain of binary rules
      ( $A \rightarrow BCD \Rightarrow A \rightarrow B Y$ ,  $Y \rightarrow C D$ ).
3. DEL : eliminate  $\epsilon$ -productions (nullable variables), patching every rule.
4. UNIT: eliminate unit productions  $A \rightarrow B$  by inlining B's rules into A.
5. Clean: remove useless/unreachable symbols.
```

(Order can vary, but DEL and UNIT must come *after* you've handled the structural shapes to avoid reintroducing them. A common safe order: START $\rightarrow$ TERM $\rightarrow$ BIN $\rightarrow$ DEL $\rightarrow$ UNIT.)

Small worked example. Convert $S \rightarrow ASA \mid aB, A \rightarrow B \mid S, B \rightarrow b \mid \epsilon$.

- After removing ϵ (B is nullable) and units, and binarizing, you get productions all of the form $X \rightarrow YZ$ or $X \rightarrow a$. (Full trace omitted for brevity; the key is following the 5 steps in order.)

3. Important Subtopics

3.1 Removing ϵ -productions (nullable variables)

- **What:** A variable is *nullable* if it can derive ϵ . Delete $A \rightarrow \epsilon$, then for every rule containing a nullable A, add versions with A present and absent.
- **Why:** CNF forbids ϵ except optionally at the start. *Interview angle:* "How do you handle ϵ in CNF?" Only $S_0 \rightarrow \epsilon$ is allowed, and only if ϵ is in the language.

3.2 Removing unit productions ($A \rightarrow B$)

- **What:** A production whose RHS is a single variable. Replace by directly giving A all of B's (non-unit) productions.
- **Why:** CNF forbids single-variable RHS. Unit productions also cause chains that slow parsing.

3.3 Removing useless symbols

- **Non-generating:** variables that can't derive any terminal string - remove them.
- **Unreachable:** variables not reachable from S - remove them.
- **Order matters:** remove non-generating *first*, then unreachable.

3.4 TERM and BIN (shaping the RHS)

- **TERM:** isolate terminals in long rules behind new variables.
- **BIN:** binarize RHS longer than 2 into cascades of binary rules.

3.5 The $2n-1$ derivation-length property

- **What:** In CNF, deriving a length-n string needs exactly $n-1$ applications of $A \rightarrow BC$ and n of $A \rightarrow a$, total $2n-1$ steps.
- **Why it matters:** Used to prove membership is decidable (finite search) and to bound the pumping lemma.

4. Real-World Example

Grammar-based membership testing / natural-language parsing. CYK parsers - used in computational linguistics and in some code/DSL validators - require the grammar in CNF. Before running CYK to answer "does this sentence/string parse?", the toolchain first normalizes the grammar to CNF. RNA secondary-structure prediction (stochastic CFGs) similarly relies on CNF-based CYK.

5. Diagrams / Mental Models

CNF production shapes (ONLY these):

$A \rightarrow B C$	(two variables)
$A \rightarrow a$	(one terminal)

$S_0 \rightarrow \epsilon$ (start only, iff ϵ in language)

CNF parse tree is essentially BINARY:



Step	Removes	Rule shape after
START	start symbol on RHS	new $S_0 \rightarrow S$
TERM	terminals inside long RHS	terminals only as $A \rightarrow a$
BIN	RHS length > 2	all RHS ≤ 2
DEL	ϵ -productions	no $A \rightarrow \epsilon$ (except S_0)
UNIT	unit rules $A \rightarrow B$	no single-variable RHS

6. Common Interview Questions

Q1. What is CNF? Every rule is $A \rightarrow BC$ or $A \rightarrow a$ (plus optional $S_0 \rightarrow \epsilon$). *Mistake:* allowing terminals mixed with variables like $A \rightarrow aB$.

Q2. Why convert to CNF? Enables CYK $O(n^3)$ parsing; gives binary parse trees and the $2n-1$ derivation bound; simplifies proofs.

Q3. List the conversion steps. START, TERM, BIN, DEL (ϵ), UNIT, remove useless. *Mistake:* removing units before ϵ (can reintroduce units).

Q4. How do you remove ϵ -productions? Find nullable variables; for each rule, add all combinations with nullable symbols omitted; delete $A \rightarrow \epsilon$.

Q5. How do you remove unit productions? For each unit $A \rightarrow B$, add all of B's non-unit rules to A; remove the unit rule.

Q6. Can CNF represent a language containing ϵ ? Only via $S_0 \rightarrow \epsilon$ where S_0 is a fresh start symbol not used on any RHS.

Q7. How many steps to derive a length- n string in CNF? Exactly $2n - 1$.

Q8. Does converting to CNF change the language? No - it's language-preserving (except possibly the trivial ϵ handling).

Q9. What are useless symbols and how are they removed? Non-generating (remove first) and unreachable (remove second) symbols.

Q10. Convert $S \rightarrow aSb \mid \epsilon$ to CNF (sketch). Handle ϵ ($S_0 \rightarrow \epsilon$, $S \rightarrow aSb|ab$), TERM ($X \rightarrow a$, $Y \rightarrow b$), BIN: $S \rightarrow X S_1$, $S_1 \rightarrow S Y$; $S \rightarrow X Y$.

7. Deep-Dive Questions

D1. Why remove non-generating symbols before unreachable ones? Removing non-generating symbols can make previously-reachable symbols unreachable; doing unreachable first could leave non-generating garbage. The order guarantees a fully reduced grammar.

D2. Can every CFG be put in CNF? Yes - every context-free language (with the ϵ caveat handled) has an equivalent CNF grammar.

D3. How much can CNF blow up the grammar size? The BIN and DEL steps can increase the number of rules polynomially (ϵ -removal can be exponential in the worst case for many nullable symbols) - a practical concern.

D4. Why does CYK specifically need CNF? Because binary rules let CYK combine two adjacent sub-spans into a larger one; the DP fills a triangular table using exactly the $A \rightarrow BC$ shape.

D5. Relationship between CNF derivation length and the pumping-lemma constant? The pumping length is related to $2^{|V|}$ (or similar), derived from the fact that a tall enough CNF parse tree must repeat a variable on a root-to-leaf path.

8. Greibach Normal Form (GNF) - Basics

1. Overview

Definition. A CFG is in **Greibach Normal Form** if every production has the form:

$A \rightarrow a \alpha$, where a is exactly one terminal and $\alpha \in V^*$ is a (possibly empty) string of **variables only**.

So each rule starts with a single terminal, followed by zero or more variables (no terminals after the first, no variable-first rules). Optionally $S \rightarrow \epsilon$ if ϵ is in the language.

Why it matters:

- **Every derivation step produces exactly one terminal.** A string of length n derives in **exactly n steps**. This makes GNF ideal for reasoning about *left-to-right, no-backtracking* recognition.
- GNF directly gives a **PDA with no ϵ -moves** (each move consumes one input symbol) - a clean top-down machine.
- Removes **left recursion** as a by-product (rules start with a terminal, so no $A \rightarrow A\alpha$).

Where used: theoretical constructions, converting grammars to real-time (ϵ -free) PDAs, some top-down parsing results.

Why interviewers ask: Usually at a *conceptual* level - "what is GNF, how does it differ from CNF, why is it useful?" Full conversion is rarely required but knowing the shape and purpose is expected.

2. Core Idea

Intuition. CNF standardizes the *tree shape* (binary). GNF standardizes the *reading order* - it guarantees every rule *emits a terminal first*, so parsing consumes input steadily, one symbol per step, with the rest of the work stored as variables.

Analogy. GNF is "productive" - every step of the derivation pays out a real output symbol immediately (like an assembly line where every station adds one finished part), whereas CNF may take steps that only rearrange variables.

Shape comparison:

CNF: $A \rightarrow BC$ or $A \rightarrow a$
 GNF: $A \rightarrow a B C D \dots$ (one terminal first, then variables)

Conversion outline (high level):

1. Convert to CNF first (or at least eliminate ϵ and unit rules).
2. Order the variables $A_1, A_2, \dots, A_n$.
3. Eliminate rules $A_i \rightarrow A_j \dots$ where $j \leq i$ (removing left recursion using the substitution + $A \rightarrow \beta A'$ trick).
4. Back-substitute so every RHS begins with a terminal.

3. Important Subtopics

3.1 The "terminal-first" shape

- **What:** RHS = one terminal then only variables.
- **Why:** Guarantees progress - each production consumes exactly one input symbol in a matching PDA.

3.2 Left-recursion elimination (key sub-technique)

- **What:** Replace $A \rightarrow A\alpha \mid \beta$ with $A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \epsilon$.
- **Why:** GNF cannot have a variable first, so all left recursion must go. This same technique is used to make grammars work with recursive-descent parsers.
- **Interview angle:** "How do you remove left recursion?" - state the $A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \epsilon$ transform.

3.3 GNF $\rightarrow$ real-time PDA (no ϵ -moves)

- **What:** Because each rule starts with a terminal, the corresponding PDA reads exactly one input symbol per move - no ϵ -transitions needed.
- **Why:** Gives a "real-time" recognizer.

3.4 Derivation length = n

- Each step outputs one terminal, so a length-n string needs exactly n leftmost-derivation steps (contrast CNF's $2n-1$).

4. Real-World Example

GNF is the theoretical justification that any CFL can be recognized by a **top-down PDA that never stalls** - relevant when building **real-time / streaming parsers** that must consume input at a steady rate (e.g. protocol parsers that act on each byte as it arrives). The left-recursion-elimination step in GNF is exactly what you must do by hand before feeding a grammar to a recursive-descent parser generator like ANTLR (older versions) - otherwise the parser infinite-loops.

5. Diagrams / Mental Models

Property	CNF	GNF
RHS shape	BC or a	a then variables
Standardizes	tree shape (binary)	reading order (terminal-first)
Derivation length for $ w =n$	$2n - 1$	n
Left recursion	may exist	eliminated
Natural PDA	needs stack manipulation	ϵ -free ("real-time")
Main use	CYK / membership	ϵ -free PDA, top-down theory

6. Common Interview Questions

Q1. What is GNF? Every rule $A \rightarrow a\alpha$ with one leading terminal followed by only variables. *Mistake:* allowing terminals after the first symbol.

Q2. GNF vs CNF? CNF fixes tree shape (binary); GNF fixes reading order (terminal-first). See table.

Q3. Why is GNF useful? One terminal per step $\Rightarrow$ length-n string in n steps; yields ϵ -free PDA; removes left recursion.

Q4. How long is a derivation of a length-n string in GNF? Exactly n steps.

Q5. Does GNF allow left recursion? No - RHS always starts with a terminal, so left recursion is impossible.

Q6. How do you eliminate left recursion? $A \rightarrow A\alpha \mid \beta$ becomes $A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \epsilon$.

Q7. Can every CFL be put in GNF? Yes (with ϵ handled separately).

Q8. What kind of PDA does GNF give? A single-terminal-per-move, ϵ -free (real-time) PDA.

Q9. Is $A \rightarrow aBc$ in GNF? No - c is a terminal after the first symbol; only variables may follow.

Q10. Which normal form does CYK use - CNF or GNF? CNF.

7. Deep-Dive Questions

D1. Why does GNF guarantee an ϵ -free PDA? Each production emits exactly one terminal, so the simulating PDA consumes one input symbol per move - never needing to move without reading.

D2. Is GNF conversion harder than CNF? Generally yes - it involves variable ordering and repeated substitution to remove left recursion, and can blow up grammar size more.

D3. Does GNF make parsing deterministic? No - GNF removes left recursion and ϵ -moves but the grammar can still require nondeterministic choice among rules $A \rightarrow a\alpha_1 \mid a\alpha_2$. Determinism needs stronger restrictions (LL/LR).

D4. Relationship between GNF and left-corner parsing? GNF's terminal-first shape aligns with left-corner / top-down predictive strategies where you commit based on the first terminal.

D5. Why do we sometimes still prefer CNF? For CYK-style dynamic programming and length/pumping analysis; the binary shape is more convenient than GNF's variable-length RHS.

9. Pumping Lemma for Context-Free Languages

1. Overview

Statement. If L is a CFL, there exists a constant p (the pumping length) such that every string $z \in L$ with $|z| \geq p$ can be split into **five** parts $z = uvwx^i y$ satisfying:

1. $|vwx| \leq p$ (the "middle window" is bounded),
2. $|vx| \geq 1$ (at least one of v, x is non-empty - you pump something),
3. $uv^iwx^iy \in L$ for all $i \geq 0$ (pump v and x *together*, any number of times).

Why it matters: It is the primary tool to **prove a language is NOT context-free**. Just as the regular pumping lemma (three parts xyz) proves non-regularity, the CFL pumping lemma (five parts, *two* pumped pieces) proves non-context-freeness. The two pumped pieces v and x reflect the PDA's stack matching *two* positions at once.

Where used: Complexity/theory proofs; showing $a^n b^n c^n$, $\{ww\}$, $\{a^i b^j c^k \mid i < j < k\}$, $\{a^{n^2}\}$ are not context-free.

Why interviewers ask: Classic "prove X is not a CFL" problems; tests rigor and understanding of *why* one stack limits a PDA.

2. Core Idea

Intuition (why five parts and why v, x pump together). A CNF parse tree for a long-enough string is *tall*; by pigeonhole some **variable A repeats** on a single root-to-leaf path. The two occurrences of A enclose a subtree. You can **cut and paste** that subtree:

- Removing it $\rightarrow i=0$ case.
- Repeating it $\rightarrow i=2,3,\dots$ cases. The repeated subtree contributes strings on **two sides** (the v on the left of the inner subtree's yield and the x on the right), which is why exactly *two* pieces (v and x) get pumped in lockstep - the grammar's recursion pumps both sides symmetrically.

Analogy. Nested Russian dolls: if a doll pattern repeats, you can insert extra copies of the middle nesting. Each inserted layer adds a matched pair (something on the left and something on the right), never just one side.

Small proof - $L = \{a^n b^n c^n \mid n \geq 0\}$ is not context-free:

1. Assume CFL with pumping length p . Choose $z = a^p b^p c^p$ ($|z| \geq p$).
2. Split $z = uvwxy$ with $|vwx| \leq p$ and $|vx| \geq 1$.
3. Since $|vwx| \leq p$, the window vwx spans **at most two** of the three symbol-blocks (it can't touch both a 's and c 's - they're p apart).
4. Pumping ($i=2$) increases counts of at most two symbols, never all three equally $\Rightarrow$ the pumped string has unequal counts $\Rightarrow$ not in L .

5. Contradiction $\Rightarrow$ L is not context-free. ■

3. Important Subtopics

3.1 The two conditions on the split

- **$|vwx| \leq p$:** confines the pumped region so it can't span too far (the crux of most proofs - it forces vx to sit within at most two blocks).
- **$|vx| \geq 1$:** ensures pumping actually changes the string.

3.2 Pumping v and x *together*

- **What:** You pump both with the same exponent i.
- **Why it differs from regular PL:** regular PL pumps a single middle piece; CFL PL pumps two, because a PDA/CNF-tree matches two positions.

3.3 The adversary game (proof structure)

- You pick p is given (adversary's), **you pick z** cleverly, adversary picks the split (subject to the two constraints), **you pick i** to force a contradiction. Choosing z well is the skill.

3.4 What it can and cannot prove

- **Can:** prove a language is *not* a CFL (necessary condition).
- **Cannot:** prove a language *is* a CFL (it's not sufficient - some non-CFLs satisfy it). For that, exhibit a CFG or PDA.

3.5 Ogden's Lemma (stronger variant, mention only)

- A refinement letting you **mark positions** that must be pumped, useful when the plain PL is too weak (e.g. proving certain languages inherently ambiguous). Good to name-drop.

4. Real-World Example

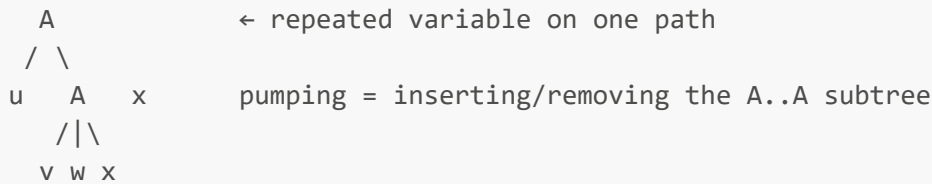
The pumping lemma is *why* you **cannot parse $a^n b^n c^n$ -style constraints or fully validate nested-with-cross-dependencies structures using a CFG alone** - e.g. checking that an XML has matching tags *and* that declared array lengths match actual counts, or that every variable is declared before use. Those "context-sensitive" checks (like type checking, or that $\langle n \rangle$ copies of something appear) are provably beyond CFGs, which is exactly why compilers have a **separate semantic-analysis phase** after parsing. The PL is the formal reason parsing and semantic checking are split.

5. Diagrams / Mental Models

$z = u \ v \ w \ x \ y$ (five parts)
 $\quad \quad \quad \underbrace{\quad \quad \quad}$
 $\quad \quad \quad |vwx| \leq p, \ |vx| \geq 1$

Pump: $u v^i w x^i y \in L$ for ALL $i \geq 0$

CNF parse tree reason:



Adversary game table:

Who	Chooses	Your goal
Adversary	pumping length p	-
You	string z ,	z
Adversary	split $u v w x y$ (with constraints)	-
You	exponent i	make $u v^i w x^i y \notin L$

6. Common Interview Questions

Q1. State the CFL pumping lemma. $z = uvwxy$ with $|vwx| \leq p$, $|vx| \geq 1$, and $uv^iwx^iy \in L$ for all $i \geq 0$. *Mistake:* using three parts (that's the regular version).

Q2. How is it different from the regular pumping lemma? Five parts vs three; pumps *two* pieces (v and x) together vs one.

Q3. Prove $a^n b^n c^n$ is not context-free. (See section 2.) *Mistake:* choosing a weak z , or forgetting the $|vwx| \leq p$ window argument.

Q4. What does $|vwx| \leq p$ buy you? It confines the pumped region to at most two adjacent blocks, so pumping can't grow all three counts.

Q5. Can the pumping lemma prove a language IS a CFL? No - only that it is NOT (necessary, not sufficient).

Q6. Prove $\{ ww \mid w \in \{a,b\}^* \}$ is not a CFL. Pick $z = a^p b^p a^p b^p$; any valid split, when pumped, breaks the two-halves-equal structure.

Q7. Why exactly two pumped pieces? Because a PDA's single stack matches two positions; the repeated CNF-tree variable yields growth on both sides.

Q8. Who chooses z and who chooses the split? You choose z ; the adversary chooses the split; you choose i .

Q9. Is $\{ a^n b^n \}$ pumpable / is it a CFL? Yes it's a CFL - the PL is satisfied (and there's a grammar). PL doesn't say it isn't.

Q10. What is Ogden's lemma? A stronger form letting you mark positions forced into the pumped region; used when plain PL is insufficient.

7. Deep-Dive Questions

D1. Derive the pumping length from the grammar. For a CNF grammar with $|V|$ variables, a string longer than $2^{|V|}$ forces a repeated variable on a root-leaf path; so $p \approx 2^{|V|}$ works.

D2. Give a non-CFL that still satisfies the pumping lemma. Such languages exist (the PL is not sufficient); this is why we say the PL only proves *non*-membership. Ogden's lemma or closure arguments handle these.

D3. Prove $\{ a^i b^j c^k \mid i < j < k \}$ is not context-free. Pick $z = a^p b^{p+1} c^{p+2}$; pumping within $\leq p$ window can't preserve all strict inequalities - contradiction.

D4. Can you sometimes prove non-CFL more easily with closure properties than the PL? Yes - e.g. intersect the language with a regular language to isolate a known non-CFL like $a^n b^n c^n$; since CFLs are closed under intersection with regular languages, a contradiction follows. Often cleaner than raw pumping.

D5. Why does the regular PL (3 parts) fail to capture CFLs? Regular PL pumps one segment (one repeated *state*); CFLs need matching of two positions (a repeated *variable*/subtree), hence two pumped segments.

8. Comparison Table

Feature	Regular Pumping Lemma	CFL Pumping Lemma
Split	$z = x y z$ (3 parts)	$z = u v w x y$ (5 parts)
Pumped pieces	one (y)	two (v and x) together
Constraint	$ xy \leq p, y \geq 1$	$ vwx \leq p, vx \geq 1$
Proves	not regular	not context-free
Underlying reason	repeated state (DFA)	repeated variable (CNF tree)

10. Closure Properties of CFLs

1. Overview

Definition. A **closure property** asks: if you apply an operation to context-free language(s), is the result still context-free? CFLs are closed under some operations and **not** under others - and knowing exactly which is a favorite exam target.

CFLs ARE closed under: Union, Concatenation, Kleene star (and +), Reversal, Homomorphism & inverse homomorphism, Substitution, and **Intersection with a Regular language**.

CFLs are NOT closed under: **Intersection** (two CFLs), **Complement**, and **Set difference** (in general).

Why it matters: Closure properties let you *build* new CFLs from known ones, and - crucially - **prove languages are NOT context-free** by contradiction (e.g. using the non-closure under intersection/complement). They're also a compact way to reason about what parsers/grammars can compose.

Why interviewers ask: The union-vs-intersection asymmetry (closed under union, NOT intersection) is a classic "gotcha," and the intersection-with-regular trick is a powerful proof technique.

2. Core Idea

Intuition for the positives. Union/concatenation/star follow directly from grammars: given grammars for L_1 , L_2 with start symbols S_1 , S_2 :

- **Union:** $S \rightarrow S_1 \mid S_2$.
- **Concatenation:** $S \rightarrow S_1 S_2$.
- **Star:** $S \rightarrow S S_1 \mid \epsilon$. Reversal reverses every RHS. These are one-line grammar constructions - that's why CFLs are closed under them.

Intuition for the negatives. Intersection would need to match *two independent constraints at once*, but a PDA has only **one stack**. Classic proof:

- $L_1 = \{ a^n b^n c^m \}$ is a CFL (match a,b; c free).
- $L_2 = \{ a^m b^n c^n \}$ is a CFL (match b,c; a free).
- $L_1 \cap L_2 = \{ a^n b^n c^n \}$ which is **NOT** a CFL (pumping lemma).
- So CFLs are **not closed under intersection**. ■

Complement follows: if CFLs were closed under complement, then by De Morgan $L_1 \cap L_2 = \neg(\neg L_1 \cup \neg L_2)$ would be a CFL (union and complement both closed) - contradicting non-closure under intersection. So **not closed under complement** either.

But CFL $\cap$ Regular = CFL. Run the PDA and the DFA *in parallel* (product construction): the finite control tracks the DFA state, the stack still does the PDA's job. One stack suffices because the regular side needs no stack.

3. Important Subtopics

3.1 Positive closures (Union, Concatenation, Star, Reversal)

- **What/why:** Direct grammar constructions (above). Always closed.
- **Interview angle:** "Give the grammar construction for union of two CFLs" - $S \rightarrow S_1 \mid S_2$.

3.2 Intersection with a Regular language (the workhorse)

- **What:** CFL $\cap$ Regular is a CFL, via PDA $\times$ DFA product.
- **Why it matters:** The **standard trick** to prove a language isn't a CFL - intersect with a regular language to strip it down to a known non-CFL.
- **Interview angle:** "Show $\{ w \mid w \text{ has equal a's, b's, c's in order } a^n b^n c^n \dots \}$ is not CF using closure."

3.3 Non-closure under Intersection and Complement

- **What:** Two CFLs' intersection need not be a CFL; complement need not be either.
- **Why:** One-stack limitation; De Morgan links the two.
- **Interview angle:** THE classic asymmetry: **union yes, intersection no**.

3.4 Homomorphism, inverse homomorphism, substitution

- **What:** Replacing symbols by strings/languages preserves context-freeness. Substitution is the "master" closure that many others derive from.

3.5 DCFLs have *different* closure (contrast)

- Deterministic CFLs **are** closed under complement (unlike general CFLs) but **not** under union or intersection. Great comparison material (see DCFL section).

4. Real-World Example

Compiler front-ends compose grammars. When a language embeds sublanguages (e.g. SQL strings inside Java, or template expressions inside HTML), the tooling relies on CFLs being closed under **substitution and concatenation** to reason about the combined syntax. Conversely, the **non-closure under intersection** is the formal reason you *cannot* enforce cross-cutting constraints (like "every opened resource is closed" *and* "counts match") purely at the grammar level - those become semantic-analysis or type-system responsibilities.

5. Diagrams / Mental Models

Grammar constructions (why positives hold):

Union: $S \rightarrow S_1 \mid S_2$

Concatenation: $S \rightarrow S_1 S_2$

Star: $S \rightarrow S S_1 \mid \epsilon$

Reversal: reverse every rule's RHS

Why intersection fails (one stack, two jobs):

$\{a^n b^n c^m\} \cap \{a^m b^n c^n\} = \{a^n b^n c^n\} \leftarrow$ not CF
 CFL CFL NOT CFL

6. Closure Summary Table

Operation	CFL closed?	DCFL closed?	Regular closed?
Union	✓	✗	✓
Concatenation	✓	✗	✓
Kleene star	✓	✗	✓
Reversal	✓	✗	✓
Intersection (two of same class)	✗	✗	✓
Intersection with Regular	✓	✓	✓
Complement	✗	✓	✓
Set difference	✗	✗	✓
Homomorphism	✓	✗	✓
Inverse homomorphism	✓	✓	✓
Substitution	✓	✗	✓

7. Common Interview Questions

Q1. Are CFLs closed under union? Yes - $S \rightarrow S_1 \mid S_2$. *Mistake:* saying no.

Q2. Are CFLs closed under intersection? No - $\{a^n b^n c^m\} \cap \{a^m b^n c^n\} = \{a^n b^n c^n\}$ (not CF). *Mistake:* confusing with union.

Q3. Are CFLs closed under complement? No - follows from non-closure under intersection via De Morgan.

Q4. Is $\text{CFL} \cap \text{Regular}$ context-free? Yes - PDA $\times$ DFA product construction. *Mistake:* thinking any intersection fails; only $\text{CFL} \cap \text{CFL}$ can fail.

Q5. Are CFLs closed under concatenation and star? Yes - $S \rightarrow S_1 S_2$ and $S \rightarrow S S_1 \mid \epsilon$.

Q6. Are CFLs closed under reversal? Yes - reverse each production's RHS.

Q7. Prove $a^n b^n c^n$ non-CF using closure. Intersect a CFL with a regular language $a^* b^* c^*$ after assuming... (or use the two-CFL intersection).

Q8. Are DCFLs closed under complement? Yes (unlike general CFLs) - swap accepting/non-accepting in the DPDA (with care over dead configs).

Q9. Why is union closed but intersection not? Union just offers a choice ($S \rightarrow S_1 \mid S_2$), needing no coordination; intersection requires *simultaneously* satisfying two stack-demanding constraints, impossible with one stack.

Q10. Are CFLs closed under homomorphism? Yes (and inverse homomorphism, and substitution).

8. Deep-Dive Questions

D1. Prove $\text{CFL} \cap \text{Regular}$ is a CFL. Product of the PDA (Q_P) and DFA (Q_D): states $Q_P \times Q_D$, stack unchanged; accept when both components accept. The DFA needs no stack, so one stack still suffices.

D2. Why is the intersection-with-regular trick so useful? It lets you carve a messy language down to a canonical non-CFL (like $a^n b^n c^n$) while staying in CFL-land, then apply the pumping lemma to the simpler residue.

D3. Are CFLs closed under difference with a regular language, $L_{\text{CF}} - L_{\text{Reg}}$? Yes - $L_{\text{CF}} - L_{\text{Reg}} = L_{\text{CF}} \cap \neg L_{\text{Reg}}$, and $\neg L_{\text{Reg}}$ is regular, so it's $\text{CFL} \cap \text{Regular} = \text{CFL}$.

D4. Since DCFLs are closed under complement but not union, what does that imply about CFL vs DCFL? It proves $\text{DCFL} \subsetneq \text{CFL}$: there's a CFL whose complement isn't a CFL, so it can't be deterministic (DCFLs would have CF complements).

D5. Are CFLs closed under the shuffle operation? No - shuffle of two CFLs need not be context-free.

11. Decision Properties of CFLs

1. Overview

Definition. A **decision property** asks whether there is an *algorithm* that always halts with a correct yes/no answer for a given question about CFGs/CFLs. Some questions are **decidable** (algorithm exists); others are **undecidable** (provably no algorithm).

Decidable for CFGs/CFLs:

- **Membership:** Is $w \in L(G)$? (CYK, $O(n^3)$.)

- **Emptiness:** Is $L(G) = \emptyset$?
- **Finiteness/Infiniteness:** Is $L(G)$ finite or infinite?

Undecidable for CFGs:

- **Equivalence:** Is $L(G_1) = L(G_2)$?
- **Ambiguity:** Is G ambiguous?
- **Universality:** Is $L(G) = \Sigma^*$?
- **Intersection-emptiness:** Is $L(G_1) \cap L(G_2) = \emptyset$?
- **Is $L(G)$ regular? / Is $L(G)$ context-free** (for a general grammar) - undecidable.

Why it matters: These results tell you the **hard limits of what tooling can do**. A parser generator *can* tell you if your grammar generates nothing (emptiness) but *cannot* in general tell you if two grammars are equivalent or if yours is ambiguous - which is why ambiguity shows up as heuristic "shift/reduce conflict" warnings rather than a definitive verdict.

Why interviewers ask: To test whether you know the decidable/undecidable boundary - a crisp, memorizable set of facts that reveals depth.

2. Core Idea

Intuition for decidable ones. They reduce to *finite searches* on the grammar:

- **Membership:** CYK fills an $O(n^2)$ table in $O(n^3)$ time using a CNF grammar - a bounded computation.
- **Emptiness:** Mark variables that can derive a terminal string (bottom-up). $L(G) \neq \emptyset$ iff S gets marked. Finite, terminates.
- **Finiteness:** Remove useless symbols, build the "variable-dependency" graph; $L(G)$ is **infinite iff that graph has a cycle** (a variable that can reach itself, generating pumpable structure). Cycle detection is decidable.

Intuition for undecidable ones. They encode the **Post Correspondence Problem (PCP)** or reduce from Turing-machine halting. E.g. one can build two CFGs whose intersection is empty *iff* a PCP instance has no solution - and PCP is undecidable, so intersection-emptiness is too. Equivalence and universality similarly reduce from undecidable problems.

Analogy. Decidable = questions you can answer by exhaustively checking a bounded structure. Undecidable = questions equivalent to solving the halting problem in disguise.

3. Important Subtopics

3.1 Membership (decidable, CYK)

- **What:** Does the grammar generate w ? Algorithm: CYK on CNF grammar, $O(n^3 \cdot |G|)$.
- **Why:** This is *parsing* - the core practical use.

3.2 Emptiness (decidable)

- **What:** Is any string generated? Mark generating variables bottom-up; check if S is generating.
- **Interview angle:** "How do you check if a grammar generates nothing?" - the marking algorithm.

3.3 Finiteness / Infiniteness (decidable)

- **What:** After removing useless symbols, detect a cycle in the variable-derivation graph; cycle $\Rightarrow$ infinite.
- **Interview angle:** Ties to the pumping lemma - infinite CFL $\Leftrightarrow$ pumpable $\Leftrightarrow$ cycle/recursion.

3.4 Equivalence, Ambiguity, Universality (UNDECIDABLE)

- **What:** No algorithm decides these for general CFGs.
- **Why it matters:** Explains real-tool limitations. *Contrast:* for **DFA**s, equivalence and universality *are* decidable - a key CFL-vs-Regular difference.
- **Interview angle:** "Is CFG equivalence decidable?" **No**. "Is DFA equivalence decidable?" **Yes**.

3.5 Intersection-emptiness (UNDECIDABLE)

- **What:** Is $L(G_1) \cap L(G_2) = \emptyset$? Undecidable (reduces from PCP).
- **Interview angle:** Contrast with single-grammar emptiness (decidable).

4. Real-World Example

Why compilers can detect "unreachable rule" but not "these two grammars are the same." A grammar tool *can* warn that a production is useless (emptiness/reachability - decidable). But you can never get a tool that reliably answers "is my refactored grammar exactly equivalent to the old one?" (equivalence - undecidable) - so grammar refactors are validated by **test suites**, not by a proof of equivalence. Likewise, "is this grammar ambiguous?" being undecidable is why bison emits **conflict warnings** (a sound but incomplete heuristic) instead of a definitive ambiguity verdict.

5. Diagrams / Mental Models

Question	CFG/CFL	DFA/Regular
Membership $w \in L$?	✓ decidable (CYK)	✓ decidable
Emptiness $L = \emptyset$?	✓ decidable	✓ decidable
Finiteness L finite?	✓ decidable	✓ decidable
Equivalence $L_1 = L_2$?	✗ undecidable	✓ decidable
Universality $L = \Sigma^*$?	✗ undecidable	✓ decidable
Ambiguity	✗ undecidable	(n/a)
Intersection empty?	✗ undecidable	✓ decidable
Is L regular?	✗ undecidable	✓ (trivially yes)

Memory hook: "Single-grammar structural questions (membership, empty, finite) = decidable. Two-grammar / equivalence / universality / ambiguity questions = undecidable."

6. Common Interview Questions

Q1. Is membership decidable for CFLs? Yes - CYK, $O(n^3)$.

Q2. Is emptiness decidable? Yes - mark generating variables; check S.

Q3. Is finiteness decidable? Yes - detect a cycle among useful variables.

Q4. Is equivalence of two CFGs decidable? No - undecidable. *Mistake:* assuming it's like DFA equivalence (which IS decidable).

Q5. Is ambiguity decidable? No - undecidable.

Q6. Is universality ($L = \Sigma^*$) decidable for CFGs? No - undecidable. (For DFAs, yes.)

Q7. Is $L(G_1) \cap L(G_2) = \emptyset$ decidable? No - undecidable.

Q8. How do you decide if $L(G)$ is infinite? Remove useless symbols, look for a cycle in the dependency graph; cycle $\Rightarrow$ infinite.

Q9. Give a decidable property that is undecidable for CFGs but decidable for DFAs. Equivalence (and universality).

Q10. Why is membership decidable but equivalence not? Membership is a bounded search (finite parse-table); equivalence requires comparing two potentially infinite languages, which reduces from PCP/halting.

7. Deep-Dive Questions

D1. Sketch the emptiness algorithm. Repeatedly mark a variable "generating" if it has a rule whose RHS consists only of terminals and already-marked variables; iterate to fixpoint; $L(G) \neq \emptyset$ iff start symbol marked.

D2. Why is CFG equivalence undecidable but DFA equivalence decidable? DFAs are closed under complement and intersection with decidable emptiness, so $L_1 = L_2$ reduces to checking $(L_1 \cap \neg L_2) \cup (\neg L_1 \cap L_2) = \emptyset$. CFLs aren't closed under complement/intersection, so that reduction fails; equivalence in fact reduces *from* PCP.

D3. Is $L(G) = \Sigma^*$ decidable for DCFLs? This connects to the famous result that **DCFL equivalence IS decidable** (Sénizergues, 1997) - a deep contrast with general CFGs. Worth mentioning for depth.

D4. Is "is $L(G)$ regular?" decidable? No - undecidable in general whether a given CFG generates a regular language.

D5. How does the finiteness test relate to the pumping lemma? An infinite CFL must contain a pumpable string (repeated variable/cycle); the cycle in the dependency graph is exactly that repeatable recursion.

12. CYK Algorithm (Cocke-Younger-Kasami)

1. Overview

Definition. CYK is a **dynamic-programming parsing / membership algorithm** that decides, for a CFG in **Chomsky Normal Form**, whether a string w of length n is generated - in $O(n^3 \cdot |G|)$ time. It also builds the parse table from which all parse trees can be recovered.

Why it matters: It is the standard *provably polynomial* answer to the membership problem for **arbitrary** CFGs (including ambiguous ones), unlike LL/LR which only work for restricted grammar classes. It is the concrete proof that "membership in a CFL is decidable in polynomial time."

Where used: natural-language parsing (constituency parsers), RNA secondary-structure prediction (stochastic CFGs, the probabilistic CYK / inside algorithm), grammar-based validators, teaching.

Why interviewers ask: It's the flagship algorithm of this topic - tests DP thinking, the CNF prerequisite, and complexity analysis in one problem.

2. Core Idea

Intuition (bottom-up substring DP). For every **substring** of w , compute the **set of variables** that can generate it. Start with length-1 substrings (single terminals), then build up: a variable A generates substring $w[i..j]$ if some rule $A \rightarrow BC$ splits it so that **B generates a left part and C generates the matching right part**. The string is in the language iff the **start symbol** generates the whole string $w[1..n]$.

Analogy. Building a wall bottom-up: first lay individual bricks (length-1), then combine adjacent bricks into 2-brick segments, then 3, ... up to the whole wall. Each higher segment is valid only if it can be split into two valid lower segments that a rule glues together.

The DP table. A triangular table $T[i][j]$ (or $T[\text{length}][\text{start}]$) where each cell holds the set of variables deriving the substring starting at position i of length j .

Recurrence:

```
Base:  T[i][1] = { A | A → w[i] is a rule }           (length-1 substrings)
Step:  for length L = 2..n, start i, try every split point k (1..L-1):
        if B ∈ T[i][k] and C ∈ T[i+k][L-k] and (A → BC) is a rule:
            add A to T[i][L]
Accept: S ∈ T[1][n]
```

Worked example. Grammar (CNF): $S \rightarrow AB \mid BC, A \rightarrow BA \mid a, B \rightarrow CC \mid b, C \rightarrow AB \mid a$. Test $w = \text{baaba}$ ($n=5$). Fill the table bottom-up; if S appears in the top cell (whole string), accept. (This is the canonical Sipser example; the mechanical fill is the exercise.)

3. Important Subtopics

3.1 CNF prerequisite

- **What:** CYK needs the grammar in CNF ($A \rightarrow BC$ or $A \rightarrow a$).
- **Why:** The binary split $A \rightarrow BC$ is what lets you combine two adjacent sub-spans; without exactly-binary rules the clean DP doesn't apply.
- **Interview angle:** "What must you do before running CYK?" - convert to CNF.

3.2 The triangular DP table

- **What:** $n(n+1)/2$ cells; cell (i, L) = variables deriving $w[i..i+L-1]$.
- **Why:** Captures all substrings once each (overlapping subproblems - classic DP).

3.3 Complexity analysis

- **Time:** $O(n^2 \text{ substrings} \times n \text{ split points} \times |G| \text{ rules}) = O(n^3 \cdot |G|)$.

- **Space:** $O(n^2 \cdot |V|)$.
- **Interview angle:** "Why is CYK $O(n^3)$?" - two loops for the span (i, L) give n^2 , inner split loop gives another n , hence n^3 (grammar size constant for fixed G).

3.4 Recovering parse trees & handling ambiguity

- **What:** Store back-pointers (which rule + split produced each variable) to reconstruct parse tree(s). Ambiguous grammars $\Rightarrow$ multiple back-pointers $\Rightarrow$ multiple trees.
- **Why:** CYK works for *any* CFG, ambiguous or not - a strength over LL/LR.

3.5 Probabilistic CYK (extension)

- **What:** With a PCFG (weighted rules), CYK finds the **most probable** parse (Viterbi-style) or total probability (inside algorithm).
- **Where:** NLP and computational biology.

4. Real-World Example

RNA secondary-structure prediction. Biologists model base-pairing (nested stems and loops) with stochastic context-free grammars because the pairing is *nested* like balanced parentheses. A **probabilistic CYK** fills the same triangular table but with probabilities, computing the most likely folding of an RNA strand in $O(n^3)$. The same DP powers **constituency parsing** of sentences in classic NLP pipelines.

5. Diagrams / Mental Models

CYK table for $w = b a a b a$ (length grows upward):

```
len 5: [ T(1,5) ] ← does S appear here? → accept/reject
len 4: [ .. ][ .. ]
len 3: [ .. ][ .. ][ .. ]
len 2: [ .. ][ .. ][ .. ][ .. ]
len 1: [ b ][ a ][ a ][ b ][ a ] ← base: variables deriving each single terminal
        1   2   3   4   5      (start position i)
```

Cell (i,L) combines: split at $k \rightarrow$ left (i,k) $\otimes$ right (i+k, L-k) via $A \rightarrow BC$

Split visualization for a length-4 span:

```
w[i..i+3] can be split as:
  [1][3]   [2][2]   [3][1]
For each split, if left ∈ T and right ∈ T and  $A \rightarrow (\text{left})(\text{right})$  exists  $\rightarrow$  A joins the
cell.
```

6. Common Interview Questions

Q1. What does CYK do? Decides membership $w \in L(G)$ (and parses) for a CNF grammar in $O(n^3)$. *Mistake:* forgetting the CNF requirement.

Q2. Why must the grammar be in CNF? Binary rules $A \rightarrow BC$ enable combining two adjacent sub-spans in the DP.

Q3. What is the time complexity and why? $O(n^3 \cdot |G|)$: n^2 spans $\times$ n split points. *Mistake*: saying $O(n^2)$.

Q4. What does cell $T[i][L]$ store? The set of variables that can derive the substring of length L starting at position i .

Q5. How do you decide acceptance? Start symbol $S \in$ top cell (whole string).

Q6. Can CYK handle ambiguous grammars? Yes - it works for any CFG; ambiguity shows as multiple derivations/back-pointers. *Mistake*: thinking it needs an unambiguous or LL/LR grammar.

Q7. Is CYK bottom-up or top-down? Bottom-up (builds from single symbols to the whole string).

Q8. How do you recover the actual parse tree? Store back-pointers (rule + split) per cell and trace from the top cell.

Q9. What's the space complexity? $O(n^2 \cdot |V|)$.

Q10. Compare CYK with Earley. Earley also parses any CFG (no CNF needed), $O(n^3)$ worst case but $O(n^2)$ for unambiguous and $O(n)$ for many practical grammars; CYK needs CNF but is simpler.

7. Deep-Dive Questions

D1. Derive the $O(n^3)$ bound precisely. Outer loops over span length L (n values) and start i (up to n) give $O(n^2)$ cells; each cell tries up to $L-1$ ($\leq n$) splits and checks each binary rule; total $O(n^3 \cdot |G|)$.

D2. How does probabilistic CYK differ? Cells store the best (max) probability of deriving the span as each variable; recurrence multiplies child probabilities by rule probability and takes the max (Viterbi) - yielding the most probable parse.

D3. Why isn't CYK used in most compilers? Compilers use restricted grammars (LL/LR) that parse in $O(n)$ linear time; CYK's $O(n^3)$ is wasteful when the grammar is deterministic. CYK shines for *general/ambiguous* grammars (NLP).

D4. Can CYK be adapted to non-CNF grammars? Not directly - you must convert to CNF first, or use Earley/GLR which handle arbitrary CFGs without CNF.

D5. What is the relationship between CYK and matrix multiplication? CFG parsing can be reduced to Boolean matrix multiplication (Valiant's algorithm), giving sub-cubic $O(n^{2.37...})$ parsing - a famous theoretical result showing parsing is "as hard as" matrix multiply.

13. Deterministic Context-Free Languages (DCFLs)

1. Overview

Definition. A **Deterministic Context-Free Language** is a language accepted by a **Deterministic Pushdown Automaton (DPDA)** - a PDA whose transition function has **at most one applicable move** for every (state,

input-or- ϵ , stack-top) configuration, with the restriction that if an ϵ -move is possible, no input-consuming move is (no choice).

Key relationships:

- **DCFL $\subsetneq$ CFL** (proper subset - some CFLs are not deterministic).
- DCFLs are **exactly the languages with LR(1) grammars** (deterministic bottom-up parseable).
- DCFLs are **precisely the CFLs that can be parsed deterministically in linear time** - which is why real programming languages are designed to be (nearly) DCFLs.

Why it matters: Every practical programming language is designed to be a DCFL (or close to it) so it can be parsed *deterministically and in linear time* by an LR/LALR parser - no backtracking, no exponential blow-up. DCFLs are where theory meets the reality of fast compilers.

Where used: the design target for every language grammar; yacc/bison/LALR parser generators accept (essentially) DCFL grammars; JSON, most of C/Java/etc.

Why interviewers ask: DCFLs sit at the intersection of theory and practice, and their **closure properties differ from general CFLs** (closed under complement, *not* under union/intersection) - a rich source of precise questions.

2. Core Idea

Intuition. A DPDA never has to *guess*. At each step, the current state + next input + stack top **uniquely determines** the move. That determinism is what makes parsing fast and unambiguous - but it also makes DPDAs strictly weaker than NPDAs, because some languages *fundamentally require* guessing.

The canonical gap - why palindromes escape DCFL.

- $L = \{ w c w^R \}$ (palindromes with an explicit center marker c) **is** a DCFL - the c tells the DPDA exactly when to switch from pushing to popping.
- $L = \{ w w^R \}$ (even palindromes, *no* marker) is a CFL but **NOT** a DCFL - without a marker the DPDA can't know where the middle is; only a *nondeterministic* PDA can guess the midpoint. This single example crystallizes $DCFL \subsetneq CFL$.

Analogy. A DPDA is like reading a book with clear chapter markers - you always know what to do next. An NPDA is like solving a maze where you may need to try paths and backtrack. Programming languages add "markers" (keywords, delimiters, semicolons) precisely to stay deterministic.

3. Important Subtopics

3.1 DPDA determinism condition

- **What:** ≤ 1 move per configuration; ϵ -moves can't compete with input moves.
- **Why:** This is the formal source of linear-time, backtrack-free parsing.

3.2 $DCFL \subsetneq CFL$ (strict inclusion)

- **What:** Every DCFL is a CFL, but not conversely (ww^R , inherently ambiguous languages).
- **Interview angle:** "Are all CFLs deterministic?" **No**.

3.3 Closure properties (different from CFLs!)

- **Closed under COMPLEMENT** ✓ (swap accept/non-accept states in the DPDA, carefully handling infinite ϵ -loops and dead states). *General CFLs are NOT.*
- **NOT closed under union** ✗, **NOT under intersection** ✗.
- **Closed under intersection with a regular language** ✓, and inverse homomorphism ✓.
- **Interview angle:** The complement result is the headline - it's how you *prove* a CFL is not deterministic: if L is a CFL but $\neg L$ is not a CFL, then L cannot be a DCFL.

3.4 DCFLs = LR(1) languages; every DCFL is unambiguous

- **What:** A language is a DCFL iff it has an LR(1) grammar. Every DCFL has an **unambiguous** grammar (determinism $\Rightarrow$ single parse).
- **Interview angle:** "Is every DCFL unambiguous?" Yes. "Is every unambiguous CFL a DCFL?" **No** (e.g. ww^R has an unambiguous grammar but isn't deterministic).

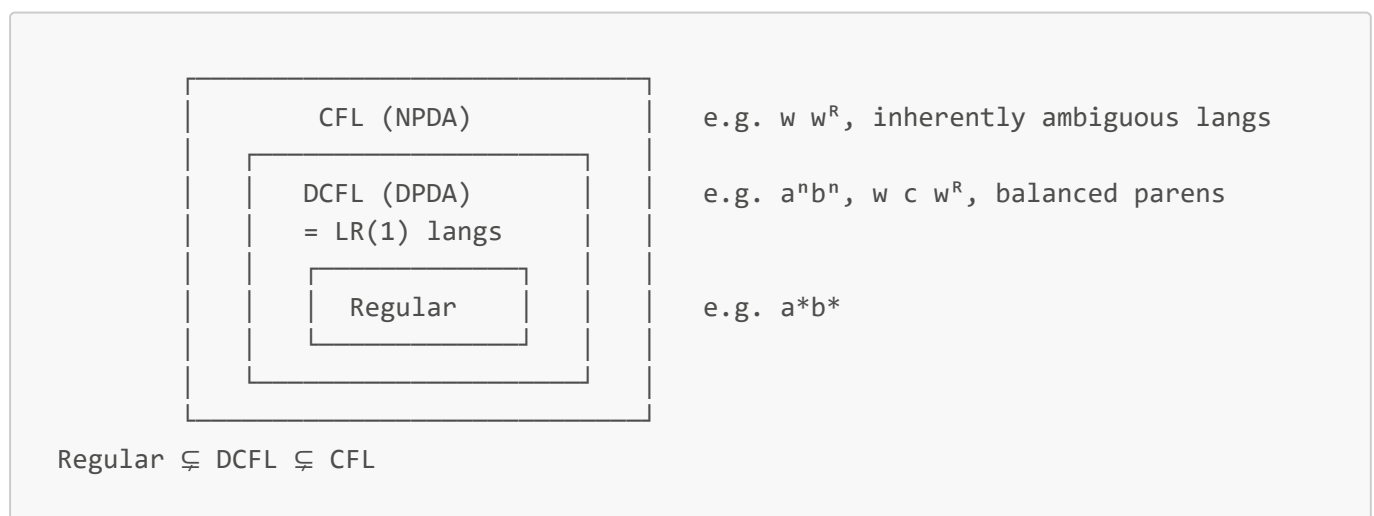
3.5 DCFL equivalence is DECIDABLE (deep contrast)

- **What:** Unlike general CFGs (equivalence undecidable), **DPDA/DCFL equivalence is decidable** (Sénizergues's theorem, 1997).
- **Interview angle:** A striking "gotcha": CFG equivalence undecidable, DCFL equivalence decidable.

4. Real-World Example

Every LR/LALR-parsed language. `yacc`, `bison`, and Java/C/C++ front-ends parse **DCFLs** because those languages were *deliberately designed* to be deterministic - keywords, delimiters, and grammar restrictions ensure the parser never guesses, giving **$O(n)$ linear-time parsing**. When a language accidentally isn't a clean DCFL (C's `typedef`-name-vs-identifier ambiguity, the "lexer hack"), compiler writers add hacks precisely because pure DCFL parsing broke down. This is the most direct "theory drives real engineering" story in the whole CFL topic.

5. Diagrams / Mental Models



6. Comparison Table: DCFL vs (general) CFL

Property	DCFL	General CFL
Machine	DPDA (deterministic)	NPDA (nondeterministic)
Grammar class	LR(1)	general CFG
Determinism = full power?	-	NPDA > DPDA (strict)
Ambiguity	always unambiguous	may be (inherently) ambiguous
Closed under complement	✓ Yes	✗ No
Closed under union	✗ No	✓ Yes
Closed under intersection	✗ No	✗ No
Closed under $\cap$ regular	✓ Yes	✓ Yes
Equivalence decidable?	✓ Yes (Sénizergues)	✗ No
Parse time	$O(n)$ linear	$O(n^3)$ (CYK, general)
Example needing it/not	$wcw^R \in \text{DCFL}$	$ww^R \in \text{CFL} \setminus \text{DCFL}$

7. Common Interview Questions

Q1. What is a DCFL? A language accepted by a deterministic PDA (≤ 1 move per configuration). *Mistake:* thinking all CFLs are deterministic.

Q2. Is DCFL a proper subset of CFL? Yes - ww^R is a CFL but not a DCFL.

Q3. Are DCFLs closed under complement? Yes (unlike general CFLs). *Mistake:* assuming CFL non-closure carries over.

Q4. Are DCFLs closed under union/intersection? No to both.

Q5. Give a CFL that is not a DCFL. $\{ ww^R \}$ (even palindromes, no center marker); also inherently ambiguous languages.

Q6. Is $\{ w c w^R \}$ deterministic? Yes - the center marker c removes the guess. Contrast with ww^R .

Q7. Is every DCFL unambiguous? Yes. Is every unambiguous CFL a DCFL? No.

Q8. What grammar class corresponds to DCFLs? LR(1) grammars.

Q9. Is DCFL equivalence decidable? Yes (Sénizergues) - unlike general CFG equivalence.

Q10. Why are real programming languages DCFLs? To allow deterministic, linear-time, backtrack-free LR/LALR parsing.

8. Deep-Dive Questions

D1. How do you prove a CFL is *not* a DCFL? Show its complement is not a CFL. Since DCFLs are closed under complement, if L were deterministic, $\neg L$ would be a CFL - contradiction. (Works for ww^R -type languages.)

D2. Why can't a DPDA recognize ww^R ? It cannot deterministically detect the midpoint; committing to "start popping now" is a guess a deterministic machine can't make correctly for all inputs.

D3. Explain complement closure for DPDAs. Convert to a DPDA that always reads the whole input (no dead ends, handle ϵ -loops), then swap final/non-final states. Care is needed so that non-accepting = truly not accepting.

D4. If DCFLs aren't closed under union, how do compilers combine sub-grammars? They don't rely on DCFL union closure; they design one integrated LR grammar. Combining two DCFL grammars can produce LR conflicts (a non-deterministic result), which is exactly the non-closure showing up in practice.

D5. Where do LL(k) languages fit? $LL(k) \subsetneq LR(k) \subseteq DCFL$. LL languages (top-down, predictive) are a subset of the deterministic (LR) languages; every $LL(1)$ language is a DCFL but not vice versa.

14. Cross-Cutting Summary & Final Cheat Sheet

14.1 The One-Page Mental Map

Regular $\subsetneq$ DCFL $\subsetneq$ CFL $\subsetneq$ CSL $\subsetneq$ Recursively Enumerable
 DFA DPDA NPDA LBA Turing Machine
 regex LR(1) CFG CSG unrestricted grammar

CFL toolbox:

Specify $\rightarrow$ CFG (BNF)

Recognize $\rightarrow$ PDA (stack)

Normalize $\rightarrow$ CNF ($A \rightarrow BC \mid a$) and GNF ($A \rightarrow a\alpha$)

Parse $\rightarrow$ CYK ($O(n^3)$, needs CNF), LL/LR (linear, DCFL)

Prove NOT CFL $\rightarrow$ CFL Pumping Lemma ($uvwxy$) or closure (n regular)

14.2 Must-Remember Facts (Interview Traps)

Trap	The correct answer
NFA = DFA, so NPDA = DPDA?	FALSE. NPDA > DPDA strictly.
CFLs closed under intersection?	No. (Closed under union though.)
CFLs closed under complement?	No. (But DCFLs are.)
CFL $\cap$ Regular?	Is a CFL (product construction).
CFG equivalence decidable?	No (undecidable). DFA equivalence: yes. DCFL equivalence: yes.
CFG ambiguity decidable?	No (undecidable).
Membership decidable?	Yes (CYK, $O(n^3)$).
Every CFL deterministic?	No. DCFL $\subsetneq$ CFL.
Every DCFL unambiguous?	Yes. Every unambiguous CFL deterministic? No.

Trap	The correct answer
CYK needs which normal form?	CNF.
GNF derivation length for $ w =n$?	n. CNF: $2n-1$.
Pumping lemma parts	Regular: 3 (xyz). CFL: 5 (uvwxy), pump v & x together.

14.3 Core One-Line Answers

- **CFG:** grammar with single-variable LHS; generates exactly the CFLs.
- **PDA:** finite automaton + one unbounded stack; recognizes exactly the CFLs.
- **CFG $\Leftrightarrow$ PDA:** a language is context-free iff an NPDA accepts it.
- **CNF:** $A \rightarrow BC \mid a$; enables CYK.
- **GNF:** $A \rightarrow a\alpha$; one terminal per step, ϵ -free PDA.
- **Pumping lemma (CFL):** long strings split as uvwxy, $|vwx| \leq p$, $|vx| \geq 1$, $uv^iwx^iy \in L \ \forall i$ - proves non-context-freeness.
- **CYK:** bottom-up DP membership test, $O(n^3)$, needs CNF.
- **DCFL:** DPDA-recognizable; LR(1); closed under complement but not union/intersection; the real target for programming-language parsing.

14.4 Practice Tasks (Do These to Master the Topic)

1. **Write grammars** for: $\{a^n b^n\}$, $\{a^n b^{2n}\}$, palindromes, balanced parentheses, $\{a^n b^m c^n\}$ (match a,c, equal a's and b's, $\{a^i b^j c^k \mid i=j \text{ or } j=k\}$ (note: inherently ambiguous).
2. **Draw two parse trees** for $id+id*id$ under $E \rightarrow E+E \mid E^*E \mid id$, then rewrite the grammar to be unambiguous and confirm one tree.
3. **Trace a leftmost and a rightmost derivation** for the same string and confirm they yield the same string but different step orders.
4. **Build a PDA** (with transition table) for $\{a^n b^n\}$ and for $\{wcw^R\}$; note which needs nondeterminism ($\{ww^R\}$ does).
5. **Convert a grammar to CNF** step by step (START $\rightarrow$ TERM $\rightarrow$ BIN $\rightarrow$ DEL $\rightarrow$ UNIT), e.g. $S \rightarrow ASA \mid aB$, $A \rightarrow B \mid S$, $B \rightarrow b \mid \epsilon$.
6. **Run CYK by hand** on $baaba$ with the CNF grammar from section 12 and check whether S appears in the top cell.
7. **Prove non-context-freeness** with the pumping lemma for $\{a^n b^n c^n\}$, $\{ww\}$, and $\{a^i b^j c^k \mid i < j < k\}$.
8. **Use closure** to prove $\{w \mid w \text{ has equal } a's, b's, \text{ and } c's\}$ is not a CFL (intersect with $a^*b^*c^*$).
9. **Fill the closure table** (CFL vs DCFL vs Regular) from memory, then verify against section 10.
10. **Explain in one breath** why real programming languages are designed as DCFLs (linear-time LR parsing, no backtracking).
11. **Eliminate left recursion** from $E \rightarrow E+T \mid T$ and $A \rightarrow A\alpha \mid \beta$ (produce the $A \rightarrow \beta A'$, $A' \rightarrow \alpha A' \mid \epsilon$ form).
12. **(Coding)** Implement CYK in Python (~30 lines): input CNF rules + string, output accept/reject and the DP table.

14.5 How to Explain CFLs Confidently in an Interview

"Context-free languages are one level above regular languages: a regular language is what a finite-memory machine (a DFA) can recognize, and a context-free language is what you get when you add a single unbounded stack - a pushdown automaton. That stack lets you count and match nested

structure, like $a^n b^n$ or balanced brackets, which regex provably can't do. We specify CFLs with context-free grammars - single variable on the left of each rule - and the CFG and the PDA are exactly equivalent in power. In practice, CFGs *are* the syntax of programming languages: the parser builds a parse tree from the grammar. Two big caveats interviewers love: nondeterministic PDAs are strictly stronger than deterministic ones - unlike finite automata where NFA equals DFA - and CFLs are closed under union but *not* intersection or complement. For fast parsing we restrict to deterministic CFLs (LR(1) grammars), which every real language is designed to be, so they parse in linear time; for general grammars we fall back to CYK, an $O(n^3)$ dynamic-programming membership test that needs the grammar in Chomsky Normal Form."

14.6 Common Mistakes Across the Whole Topic

1. Assuming **NPDA = DPDA** (carrying over NFA=DFA). They are **not** equal.
2. Confusing **union (closed)** with **intersection (not closed)** for CFLs.
3. Thinking **complement of a CFL is a CFL** - it isn't in general (but is for DCFLs).
4. Treating **ambiguity as a property of the language** rather than the grammar (except inherently ambiguous languages).
5. Using the **3-part regular pumping lemma** on a CFL problem instead of the **5-part** CFL version; or pumping only one of v/x instead of both together.
6. Forgetting **CYK requires CNF** and quoting $O(n^2)$ instead of **$O(n^3)$** .
7. Claiming **CFG equivalence/ambiguity is decidable** - both are **undecidable**.
8. Writing a grammar for $\{a^n b^n\}$ that allows unequal counts, or forgetting the ϵ /base case.
9. Believing a **single stack** can match two independent constraints ($a^n b^n c^n$) - it can't; that's the pumping-lemma intuition.
10. Mixing up **CNF ($A \rightarrow BC \mid a$, binary tree, $2n-1$ steps)** with **GNF ($A \rightarrow a\alpha$, terminal-first, n steps)**.

14.7 Final Cheat Sheet (Grab-and-Go)

Concept	One-liner
CFG	4-tuple (V, T, P, S) ; LHS is one variable; generates CFLs
Derivation	rule-application sequence $S \Rightarrow^* w$; LMD=leftmost, RMD=rightmost
Parse tree	structure of a derivation; ambiguity = ≥ 2 trees for one string
Ambiguity	grammar property; fix via precedence/associativity layering; undecidable in general
PDA	DFA + one stack; recognizes CFLs; NPDA > DPDA
CFG $\Leftrightarrow$ PDA	context-free iff some NPDA accepts it
CNF	$A \rightarrow BC$ or $A \rightarrow a$; enables CYK; $ w =n \rightarrow 2n-1$ steps
GNF	$A \rightarrow a\alpha$; one terminal/step; ϵ -free PDA; $ w =n \rightarrow n$ steps
Pumping lemma	$uvwx$, $ vwx \leq p$, $ vx \geq 1$, pump v & x together; proves NOT context-free
Closure 	union, concat, star, reversal, n regular, homomorphism
Closure 	intersection, complement, difference

Concept	One-liner
Decidable	membership, emptiness, finiteness
Undecidable	equivalence, ambiguity, universality, $\emptyset$ -emptiness
CYK	$O(n^3)$ bottom-up DP membership test; needs CNF; parses any CFG
DCFL	DPDA/LR(1); $\subsetneq$ CFL; closed under complement, NOT union/intersection; linear-time parse; the real-world target
Most-asked	prove not-CFL (pumping/closure), disambiguate expression grammar, CFG $\rightarrow$ PDA, run CYK, NPDA vs DPDA

End of guide. Study order suggestion: CFG $\rightarrow$ Derivations $\rightarrow$ Parse Trees $\rightarrow$ Ambiguity $\rightarrow$ PDA $\rightarrow$ CFG $\leftrightarrow$ PDA $\rightarrow$ CNF $\rightarrow$ CYK $\rightarrow$ Pumping Lemma $\rightarrow$ Closure $\rightarrow$ Decision $\rightarrow$ GNF $\rightarrow$ DCFL, then drill the cheat sheet and the "must-remember traps."